DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA0607 Design and Analysis of Algorithms

ASSIGNMENT 2 – CO2

Brute-Force and Exhaustive-Search Techniques

Sorting · Sequential Search · String Matching · Closest Pair · Convex Hull · Exhaustive Search · Day-4 Design Challenge

Complete Assignment Report

Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate

Item	Details
Student name	RITHIKA G
Register number / Section	192571027
Course Outcome	CO2 – Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems
Nature / Marks	Individual · 50 marks
GitHub repository	https://github.com/192571027/CSA0607-
Implementation language	Python 3.12 (matplotlib for graphs)

Contents and evidence map

Question	Topic	Marks	Code	Pseudocode	Raw data
Q1	Sorting under different data conditions	6	q1_sorting.py	q1_pseudocode.md	q1_*.csv
Q2	Sequential search – engineering boundary	4	q2_sequential_search.py	q2_pseudocode.md	q2_*.csv
Q3	Brute-force string matching behaviour	6	q3_string_matching.py	q3_pseudocode.md	q3_*.csv
Q4	Brute-force closest pair	6	q4_closest_pair.py	q4_pseudocode.md	q4_*.csv
Q5	Brute-force convex hull from geometry	7	q5_convex_hull.py	q5_pseudocode.md	q5_*.csv
Q6	Exhaustive search & combinatorial growth	8	q6_exhaustive_search.py	q6_pseudocode.md	q6_growth.csv
Q7	Day-4 Design Challenge (closest-pair redesign)	13	q7_redesign_closest_pair.py	q7_pseudocode.md	q7_*.csv

Method note. For every question the operation counts (comparisons, moves, pairs, distance calculations, side tests, candidates) are the primary evidence; execution time is reported only as

supporting evidence. Predictions were derived from the algorithm structure and recorded in the program output before the measurements were printed. Where a prediction turned out to be wrong, this is stated openly in the report. All input data are produced by seeded random generators, so every count in this report is exactly reproducible (times vary slightly between runs and machines).

Environment. Interpreted Python 3.12 on a cloud container. All time-based limits (response-time limits, maximum n) therefore apply to this environment; compiled code would run roughly 50–100× faster, which shifts those limits but never changes the growth laws.

Q1 – Sorting Under Different Data Conditions

1.1 Problem and question

A fulfilment centre sorts orders by dispatch-priority score. Should Selection, Bubble or Insertion Sort always be preferred, or does the choice depend on the input? About 95% of existing orders are already in order, and small batches of new orders arrive periodically.

1.2 Predictions made before implementation (with algorithmic reasons)

Input condition	Selection Sort	Bubble Sort (early exit)	Insertion Sort
Already sorted	$n(n-1)/2$ comparisons – scans the whole unsorted part every pass; 0 swaps	$n-1$ comparisons (one pass, no swap → stop)	$n-1$ comparisons, 0 shifts
Reverse sorted	$n(n-1)/2$ comparisons, only $\lfloor n/2 \rfloor$ swaps	$n(n-1)/2$ comparisons and swaps (worst case)	$n(n-1)/2$ comparisons and shifts (worst case)
Random	$n(n-1)/2$ comparisons, $<n$ swaps	$\approx n^2/2$ comparisons, swaps = inversions $\approx n^2/4$	$\approx n^2/4$ comparisons and shifts
95% sorted + small batch appended (D1)	Unchanged: $n(n-1)/2$ – not adaptive	Each appended order moves left ONE place per pass → passes ≈ distance travelled → close to n passes	Cost = $(n-1)$ + inversions; only the few appended orders move → cheap
All priorities equal (edge)	$n(n-1)/2$ comparisons, 0 swaps	$n-1$ comparisons, 0 swaps	$n-1$ comparisons, 0 shifts

Exact formulae used for the predictions: Bubble moves = Insertion shifts = number of inversions; Bubble passes = 1 + max(number of larger elements to the left of any element) because an element can move left only one position per pass; Insertion comparisons = inversions + $(n-1)$ − z, where z is the number of elements smaller than everything before them. The program computes these from an independent inversion count before running any sort.

1.3 Test data design (n = 1000)

ID	Dataset	Purpose
D1	950 existing orders already sorted + 50 NEW orders with random priority appended at the end	Contrasting dataset 1 – the real scenario (95% ordered)
D2	Strictly reverse-sorted priorities	Contrasting dataset 2 – significantly disordered, worst case
D3	All 1000 orders have the same priority	Edge case of my own – ties; tests stability and adaptiveness
D1b	Sorted, then 5% of orders displaced by ≤ 5 positions	Shows that 'nearly sorted' depends on WHERE the disorder is
D0 / D0b	Random / already sorted	Baselines
Stability	1000 orders, only 5 distinct priorities	Checks whether equal-priority orders keep arrival order

1.4 Algorithms (pseudocode summary)

InsertionSort(A): for i←1..n-1: v←A[i]; j←i-1
while j≥0 and A[j]>v: A[j+1]←A[j]; j←j-1 // count 1 comparison per test
A[j+1]←v

BubbleSort(A): repeat passes: swap adjacent out-of-order pairs; stop when a pass makes no swap

SelectionSort(A): for i←0..n-2: find index m of the minimum in A[i..n-1]; swap A[i],A[m] if m≠i

Full pseudocode: Q1_Sorting_Algorithms/Pseudocode/q1_pseudocode.md. Counted: comparisons, moves (swaps/shifts), time (best of 3).

1.5 Results – operation counts (n = 1000)

Dataset	Algorithm	Comparisons	Moves	Predicted comp.	Pred. check	Time ms	Stable*
D0 Random	Selection	499,500	992	499,500	OK	24.73	no
D0 Random	Bubble	498,275	254,248	498,275	OK	49.90	yes
D0 Random	Insertion	255,239	254,248	255,239	OK	22.73	yes
D0b Already sorted	Selection	499,500	0	499,500	OK	23.87	yes
D0b Already sorted	Bubble	999	0	999	OK	0.05	yes
D0b Already sorted	Insertion	999	0	999	OK	0.09	yes
D1 Nearly sorted	Selection	499,500	994	499,500	OK	24.22	no
D1 Nearly sorted	Bubble	499,380	24,758	499,380	OK	29.35	yes
D1 Nearly sorted	Insertion	25,757	24,758	25,757	OK	2.37	yes
D1b Nearly sorted	Selection	499,500	48	499,500	OK	24.50	no
D1b Nearly sorted	Bubble	6,972	205	6,972	OK	0.40	yes
D1b Nearly sorted	Insertion	1,203	205	1,203	OK	0.10	yes
D2 Reverse sorted	Selection	499,500	525	499,500	OK	25.23	no
D2 Reverse sorted	Bubble	499,500	499,442	499,500	OK	61.34	yes
D2 Reverse sorted	Insertion	499,498	499,442	499,498	OK	44.65	yes
D3 EDGE CASE	Selection	499,500	0	499,500	OK	24.11	yes

D3 EDGE CASE	Bubble	999	0	999	OK	0.06	yes
D3 EDGE CASE	Insertion	999	0	999	OK	0.08	yes

**Stable on that particular dataset (the dedicated stability test is below). Observed Bubble passes equalled the prediction in every case (e.g. 984 on D1, 7 on D1b, 999 on D2, 1 on sorted/equal data).*

Stability test (5 priority levels): Selection sorted=True, stable=**False**; Bubble stable=True; Insertion stable=True.

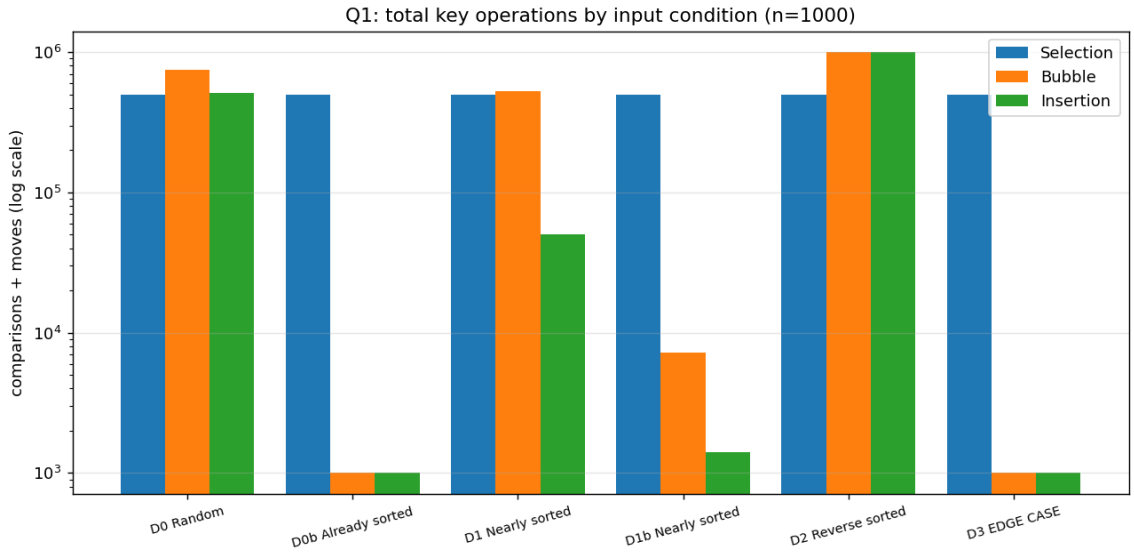


Figure 1.1 – Total key operations (comparisons + moves) by input condition, log scale

1.6 Scaling for the real scenario (95% ordered + 5% appended batch)

n	Selection comps	Bubble comps	Insertion comps	Sel/Ins	Time ms (Sel / Bub / Ins)
250	31,125	31,020	1,270	24.5×	1.3 / 1.4 / 0.1
500	124,750	124,189	5,437	22.9×	5.9 / 6.5 / 0.5
1000	499,500	498,275	27,314	18.3×	24.4 / 29.0 / 2.6
2000	1,999,000	1,998,895	97,937	20.4×	100.8 / 118.9 / 9.1
3000	4,498,500	4,494,314	223,195	20.2×	226.5 / 290.7 / 21.1

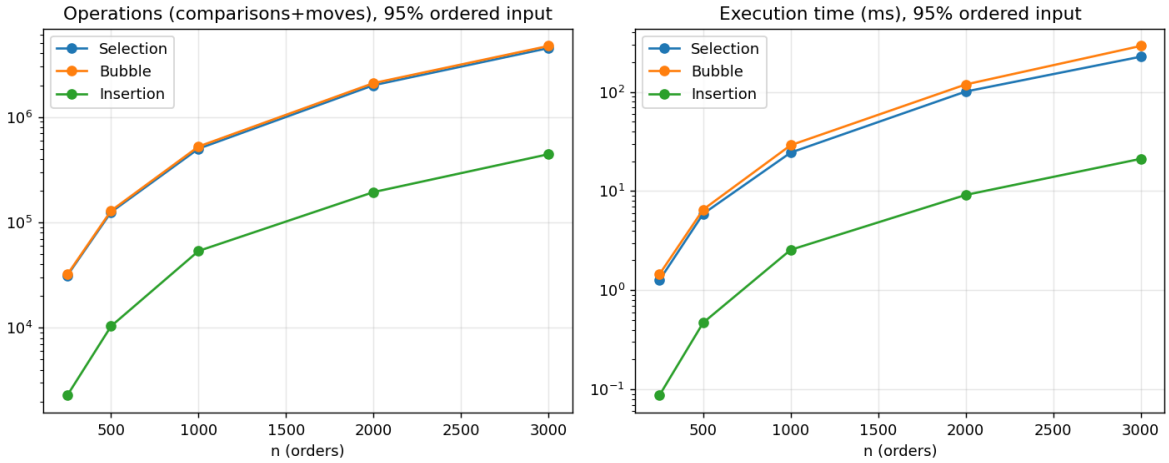


Figure 1.2 – Growth for the 95%-ordered scenario

1.7 Prediction versus observation

- **All predicted comparison counts matched the observed counts exactly (18/18)**, and Bubble/Insertion moves equalled the inversion count. The reasoning about how each algorithm works is therefore validated, not just the trend.
- **Selection Sort** performed $n(n-1)/2 = 499{,}500$ comparisons on every input – even on sorted data – confirming it is not adaptive. It has the fewest moves (e.g. 525 swaps on the reverse-sorted data against ≈499,000 for the other two), which matters only when a move is expensive.
- **Insertion Sort** is strongly adaptive: 999 comparisons on sorted/equal data, 25,757 on D1 (19× fewer than Selection) and 1,203 on D1b.
- **Bubble Sort was the surprise in magnitude.** On D1 it needed 984 passes and 499,380 comparisons – almost as many as Selection Sort – although 95% of the data was in order. The appended low-priority orders must travel far to the left and Bubble Sort moves an element left only one place per pass. On D1b (disorder only local) it needed just 7 passes and 6,972 comparisons. The lesson: "nearly sorted" is not one property – the position and distance of the disorder decide the cost.
- On the reverse-sorted data Bubble and Insertion reach their worst case ($\approx n^2/2 = 499{,}500$ comparisons) – the prediction table is confirmed.
- **Edge case D3:** with all-equal priorities Bubble and Insertion cost $n-1$ comparisons and no moves, while Selection still costs 499,500. Because Selection Sort swaps non-adjacent elements it is not stable (stability test: False), which would reorder equal-priority orders – unacceptable for first-come-first-served dispatch.

1.8 Design decision – recommendation

Recommendation: Insertion Sort. Not "one algorithm always": for random or reversed data all three are quadratic and the choice would be driven by move cost and stability. For this fulfilment centre the evidence is specific:

- Cost is (n−1) + inversions. With 95% of orders in place only the new batch creates inversions: at n = 3000 Insertion needs 223,195 comparisons against 4,498,500 for Selection and 4,494,314 for Bubble (≈20×), and runs ≈10× faster.
- It is stable (equal-priority orders keep arrival order), in place (O(1) memory) and online – each newly arriving order can be inserted at a cost equal to its displacement.
- Bubble Sort is rejected because of the observed 'small element at the end' behaviour (984 passes); Selection Sort because it ignores existing order and is unstable.
- **Limits of the recommendation:** its advantage disappears if batches become large or arrive in reverse priority order; the cost is k·n/2 shifts for a batch of k orders. Monitor the inversion count; if batches grow beyond roughly a quarter of the list, a sort with O(n log n) behaviour (outside the scope of CO2) or sorting the batch and merging should be considered.

Q2 – When Is Sequential Search an Appropriate Engineering Choice?

2.1 Design and prediction

Records (Tool ID, Technician ID, Issue Time, Work Bay) are held in an unordered list; the search key is the Tool ID. Prediction: a successful search for the key at position p costs exactly p comparisons (best 1, worst n, average over all keys (n+1)/2); an unsuccessful search costs exactly n. If a fraction s of requests succeed, the expected cost is s(n+1)/2 + (1−s)n.

```
SequentialSearch(R, key): comps←0
    for i←0..n-1: comps←comps+1 ; if R[i].ToolID = key then return (i, comps)
    return (-1, comps)
```

2.2 Experiments and comparison-count evidence

n	first	25%	middle	75%	last	unsuccessful	avg all keys (measured)	(n+1)/2
100	1	26	51	76	100	100	50.5	50.5
1,000	1	251	501	751	1,000	1,000	500.5	500.5
10,000	1	2,501	5,001	7,501	10,000	10,000	5000.5	5000.5
100,000	1	25,001	50,001	75,001	100,000	100,000	50412.7	50000.5

(For n = 100,000 the average is taken over 2,000 random keys, hence 50,412.7 against 50,000.5.) The measured counts equal the predictions exactly.

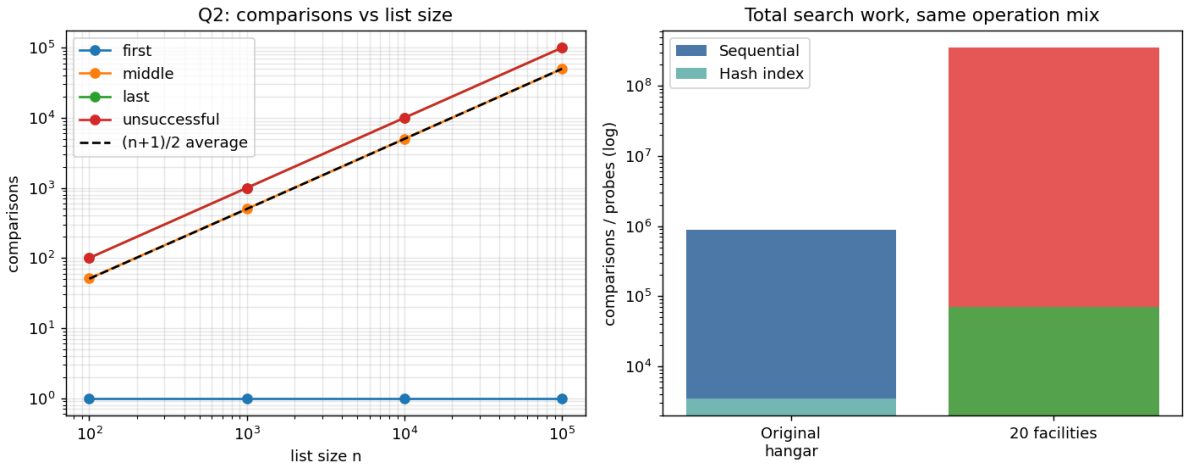


Figure 2.1 – Left: comparisons against list size (log–log). Right: total work for the same operation mix, before and after growth.

- **Successful searches** cost the position of the record: 1 (first) … n (last); averaged over all keys (n+1)/2. Cost is linear in n.
- **Unsuccessful searches** always cost the full n – the worst case – because absence can only be established by examining every record. If half the requests are for tools that are not currently issued, the expected cost rises from 0.50n to 0.75n; at 10% success it is 0.95n.

2.3 Operating boundary

Acceptability criteria I defined. C1: one worst-case lookup must complete within 50 ms (feels instant on a technician's screen) → $n \leq 0.05 \cdot r$. C2: all lookups together must use at most 10% of one core, leaving headroom → $q \cdot (n/2) \leq 0.10 \cdot r$, i.e. $q \cdot n \leq 0.2 \cdot r$. Measured speed of this environment: $r \approx 22.4$ million comparisons/second (interpreted Python).

So C1 allows n up to ≈ 1,121,500 records (one query is never the problem), and C2 gives $\mathbf{q \cdot n \leq \approx 4.5 \times 10^6}$: e.g. n = 500 → up to ≈ 9,000 lookups/s; n = 10,000 → ≈ 450/s; n = 100,000 → ≈ 45/s; n = 1,000,000 → ≈ 4.5/s.

n \ lookups per second	0.01	0.1	1	10	100	1000
100	0.000	0.000	0.000	0.002	0.022	0.223
1,000	0.000	0.000	0.002	0.022	0.223	2.229
10,000	0.000	0.002	0.022	0.223	2.229	22.290*
100,000	0.002	0.022	0.223	2.229	22.290*	222.899*
1,000,000	0.022	0.223	2.229	22.290*	222.899*	2228.992*

*Core utilisation (%) caused by lookups; * exceeds the 10% budget.*

Answer. Sequential search is a reasonable choice when (i) the list is small enough that q·n stays below the budget (hundreds to a few thousand records with at most a few lookups per second), (ii) the list is unordered and changes constantly, so keeping a sorted order or an index would cost more than the searches it saves (issue = append, return = search + delete, no maintenance), and (iii) simplicity and reliability matter. I stop recommending it when the **product** q·n of records and request rate exceeds about 4.5×10^6 per second in this environment (or ~50× more if compiled), or when most searches are unsuccessful and latency is critical – whichever comes first. Dataset size alone or frequency alone is not the criterion; their product is.

2.4 Changed condition – 20 maintenance facilities

Assumptions (explicit, replaceable by real figures): one hangar ≈ 500 tools issued and ≈ 5,000 operations/day; 20 facilities centralised ≈ 10,000 tools and ≈ 100,000 operations/day; mix 40% lookup, 30% issue, 30% return (a return must first find the record). The same operation mix was simulated for a sequential list and for a hash index keyed on Tool ID.

Scenario	Tools in service	Operations	Sequential comparisons	Hash probes
one hangar	500	5,000	888,995	3,502
20 facilities (centralised)	10,000	100,000	351,762,536	69,915

- Records ×20 and requests ×20 make the sequential cost grow ×396 (≈ 20×20), because every request scans a list that is itself 20× longer. The hash index grows only ×20 (with requests).
- In absolute terms, 3.5×10^8 comparisons/day ≈ 16 s of CPU per day, and peak utilisation (10× the average rate) is ≈ 0.43% of one core against my 10% budget. **So, honestly, with these assumptions the original recommendation still passes both criteria**, with about a 23× margin left.
- **Re-evaluated recommendation:** (a) If every facility keeps its own list, n and q per list are unchanged and sequential search remains appropriate. (b) For one central system I would replace it by a hash index on Tool ID: it costs one dictionary, supports O(1) issue/return/lookup and removes the q·n scaling, so a further ≈ 20× growth (or much burstier automated lookups such as barcode scans at every issue and return) cannot break the response-time criterion. (c) A sorted array with binary search is not a good fit because the list changes continuously (O(n) insertions).

Q3 – Engineering the Behaviour of Brute-Force String Matching

3.1 Algorithm and cost model

```
BruteForceMatch(T[0..n-1], P[0..m-1]):  comps←0
   for i←0..n-m:                 // n-m+1 alignments
      j←0
      while j<m: comps←comps+1 ; if T[i+j]≠P[j] break ; j←j+1
      if j=m: report match at i
```

Work at an alignment = (length of the matching prefix) + 1, at most m. Total work therefore depends on how long partial matches are, not on n and m alone.

3.2 Designed tests and predictions (made before execution; n = 10,000, m = 8)

Test	Text / pattern	Predicted	Observed	Obs/Pred	Matches
T1 BEST-LIKE	text a^n, pattern 'Xerror!!' (first char never in text)	9,993	9,993	1.000	0
T2 ENGINEERED	text A^n, pattern A^(m-1)B (repeated partial matches)	79,944	79,944	1.000	0
T3 OWN CHOICE	realistic synthetic log, signature 'password'	10,155	10,302	1.014	25
T4 EXTRA	pattern equals first m chars of the text (match at start)	8	8	1.000	1

Test	Why the prediction was made
T1 (low work)	Pattern starts with ‘X’, which never occurs in aaaa…a: every alignment fails on the FIRST comparison → n−m+1 = 9,993.
T2 (engineered)	Text A^n, pattern A^7B: every alignment matches 7 characters and fails on the 8th → m per alignment → (n−m+1)·m = 79,944.
T3 (my choice)	Realistic synthetic log, signature ‘password’. Expected cost per alignment E = 1 + f(p0) + f(p0)f(p1) + … from the character frequencies of the log text → 10,155.
T4 (extra)	Pattern equals the first 8 characters; algorithm stops at the first match → exactly m = 8 comparisons.

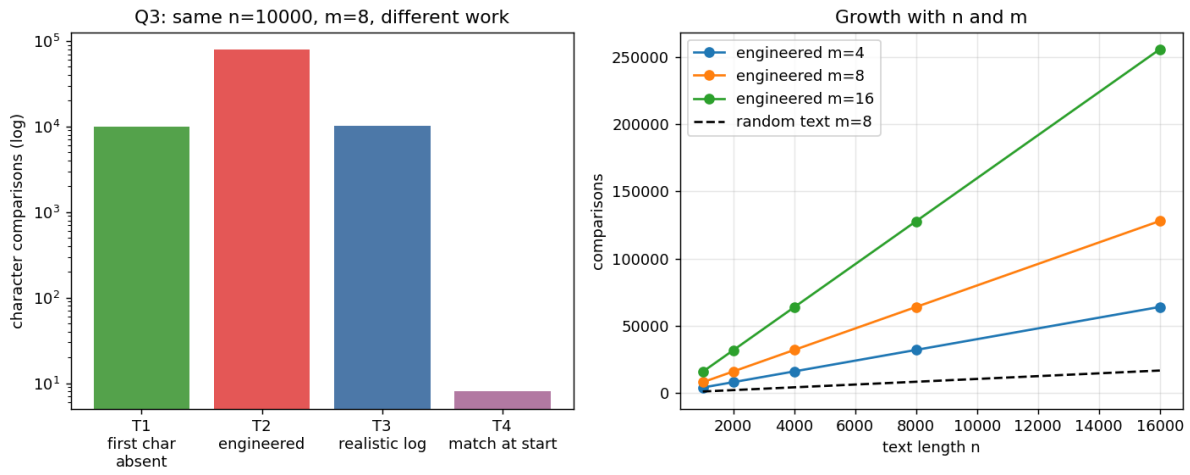


Figure 3.1 – Left: identical n and m, very different work. Right: growth of the engineered case with n and m.

3.3 Additional experiments

Alphabet size (random texts, n = 10,000, m = 8): comparisons per alignment were 1.986 (2 symbols), 1.332 (4), 1.143 (8), 1.038 (26) and 1.016 (62); observed totals agreed with the frequency-based prediction to within 0.1%. **Scaling of the engineered case**: comparisons equalled (n−m+1)·m exactly for all 15 combinations of n (1,000–16,000) and m (4, 8, 16); on random lowercase text the same n and m cost ≈ n comparisons regardless of m (e.g. 16,612–16,657 for n = 16,000).

3.4 Inquiry – prediction versus experiment and explanation

- T1, T2 and T4 matched their predictions exactly. T3 was 1.4% above the prediction (10,302 vs 10,155): the independence assumption behind E ignores that the signature genuinely occurs 25 times (25 × 8 = 200 comparisons) and that words in a log repeat. The model is good but not exact for real text.
- **Why does one input need much more work?** T1 and T2 have the same n and m, yet T2 costs 8.0× more (79,944 vs 9,993; 8.00 vs 1.00 comparisons per alignment). In T1 each alignment is rejected immediately; in T2 each alignment re-reads m−1 characters that the previous alignment already matched.
- **Characteristics that create repeated comparisons:** (1) a pattern whose prefix is repetitive and resembles the text (long partial matches), (2) a text that is itself repetitive / low-entropy, (3) a small alphabet (binary text costs ≈ 2 per alignment, 62-symbol text ≈ 1.02), and (4) a mismatch located at the END of the pattern. After a mismatch the algorithm shifts by exactly one and discards everything it learned, so the same text character may be compared up to m times.

- **What this shows about evaluating an algorithm by input size only:** n and m do not determine the work; it ranges from $n-m+1$ to $(n-m+1)\cdot m$ for identical sizes. Big-O worst case $\Theta(nm)$ describes T2, while typical log text behaves like $\Theta(n)$. An honest evaluation needs the input distribution and a bound on the worst case. In a security setting this matters twice: average behaviour is acceptable (≈ 1.03 comparisons/char) but an attacker who can influence log content could craft repetitive input and force the worst case, so the system must be tested with adversarial data and not only with typical logs.

Q4 – Closest Pair and the Cost of Examining Every Possibility

4.1 Design

- **Candidate pairs:** every unordered pair {Di, Dj}.
- **Avoiding duplicates:** (Di, Dj) and (Dj, Di) have the same distance, so the inner loop starts at j = i+1 (and i ≠ j is automatic). An ordered i ≠ j scan would examine 999,000 pairs for n = 1000 instead of 499,500.
- **Distance:** squared Euclidean distance dx^2+dy^2 – it has the same minimiser as the true distance, so the square root is avoided.

```
BruteForceClosestPair(D): best←∞
  for i←0..n-2: for j←i+1..n-1:
    d←(xi-xj)²+(yi-yj)² ; pairs←pairs+1 ; calcs←calcs+1 ; if d<best: best←d,(bi,bj)←(i,j)
```

4.2 Mathematical derivation of the number of pairs

Drone 0 is paired with n−1 later drones, drone 1 with n−2, …, drone n−2 with 1, drone n−1 with none:

$$S = (n-1) + (n-2) + \ldots + 2 + 1 = \Sigma_{k=1}^{n-1} k = n(n-1)/2 = C(n,2) = n!/(2!(n-2)!)$$

Check by pairing: writing the sum forwards and backwards gives n−1 columns each summing to n, i.e. 2S = n(n−1). Hence pairs = $\Theta(n^2)$: $C(2n,2)/C(n,2) = 2(2n-1)/(n-1) \rightarrow 4$ as n grows, i.e. doubling n quadruples the pairs (observed 4.001–4.008).

4.3 Experimental results (random drones in a 10 km × 10 km airspace)

n	Pairs examined	Distance calculations	n(n−1)/2	Time (s)	Closest distance (m)
100	4,950	4,950	4,950	0.0006	71.901
250	31,125	31,125	31,125	0.0029	12.890
500	124,750	124,750	124,750	0.0120	12.358
1,000	499,500	499,500	499,500	0.0503	0.320
2,000	1,999,000	1,999,000	1,999,000	0.2009	3.127
4,000	7,998,000	7,998,000	7,998,000	0.8440	2.006
8,000	31,996,000	31,996,000	31,996,000	3.5835	0.839

Pairs examined = distance calculations = n(n−1)/2 for every n (exact equality, prediction confirmed). Pair counts multiplied by 4.001–4.008 for each doubling of n (theory: → 4); time multiplied by 3.6–4.4. A least-squares fit t(n) = c·n² gives c = 5.58e-08 s and the log–log slope of the measured times is 1.990 (theory 2). A correctness check against an independent itertools.combinations minimum passed for n = 2, 3, 10, 50, 200.

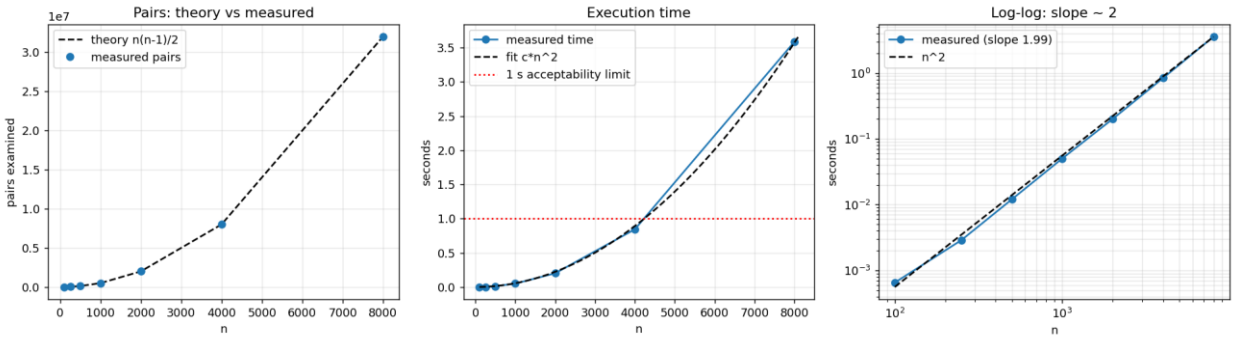


Figure 4.1 – Pairs (theory vs measured), execution time with fit and 1-second limit, and log–log growth

4.4 Scalability inquiry

Acceptable response time (my definition): 1.0 second – the monitoring system receives a new position report every second, so a collision check that takes longer yields stale results. With the fitted model, t(n) ≤ 1 s up to **n ≈ 4,234 drones** in this environment; the measured 4,000 drones took ≈0.8 s and 8,000 drones ≈3 s, so the method becomes impractical between 4,000 and 5,000 drones. Extrapolation: 10,000 drones → ≈5.5 s; 50,000 → ≈2 min; 100,000 → ≈9 min. A compiled implementation ~50× faster would raise the limit only by √50 ≈ 7× (to ≈ 30,000 drones) because the cost is quadratic.

Where the inefficiency originates. All pairs are examined, although a pair can only matter if its drones are close. Nearly all of the n(n−1)/2 pairs are far apart and can never be the answer, yet each costs a full distance evaluation. The brute-force method uses no geometric information (position, ordering) and never uses what is already known (the best distance so far) to discard other pairs. A redesigned strategy must (a) avoid comparing pairs that cannot beat the current best, by exploiting ordering or locality of the data, and (b) pay for that with a cheap preprocessing step – this is exactly the Q7 design challenge.

Q5 – Deriving a Convex Hull Algorithm from a Geometric Principle

5.1 From the geometric principle to a decision rule (my derivation)

- **Two points define a line.** For Pi = (x1, y1) and Pj = (x2, y2): a·x + b·y = c with a = y2−y1, b = x1−x2, c = x1·y2 − y1·x2.
- **Position of another point.** For Pk = (x, y) compute s = a·x + b·y − c. This equals the cross product (Pj−Pi)×(Pk−Pi): s > 0 → left of the directed line Pi→Pj, s < 0 → right, s = 0 → on the line. With integer coordinates the test is exact (no rounding errors).
- **Examining the rest:** compute s for every other point Pk (n−2 tests).
- **Condition for a hull edge:** all non-zero values of s have the same sign. Points with s = 0 (collinear) are allowed only if they lie within the segment PiPj, so that points in the middle of an edge are not mistaken for corners and a longer collinear stretch is not split.
- **Constructing the boundary:** if all s > 0, the whole set is on the left of Pi→Pj, so this is a counter-clockwise edge: succ[i] = j (if all s < 0 use succ[j] = i). Starting at the lexicographically smallest vertex and following succ[] reads the polygon in counter-clockwise order. Duplicate stations are removed first.

```
for i←0..n-2: for j←i+1..n-1:
   pos←neg←false ; ok←true
   for each k≠i,j: s←Side(Pi,Pj,Pk) ; tests++
       s>0 → pos←true ; s<0 → neg←true ; s=0 and Pk outside segment PiPj →
ok←false,break
       if pos and neg → ok←false,break
   if ok: (pos ? succ[i]←j : neg ? succ[j]←i : both directions)  // all-collinear case
follow succ[] from the smallest vertex
```

5.2 Test configurations and visualisation

Test	Configuration	Distinct n	Hull vertices	Side tests (early exit)	Side tests (no early exit)
T1 mixed	6 boundary + 14 interior stations	19	6	1,088	2,907

T2 CHALLENGE	collinear points on edges, duplicates, interior diagonal	13	4	381	858
T3 DEGENERATE	all stations collinear	8	2	48	168
T4 WORST CASE	16 stations on a circle (all are hull vertices)	16	16	874	1,680

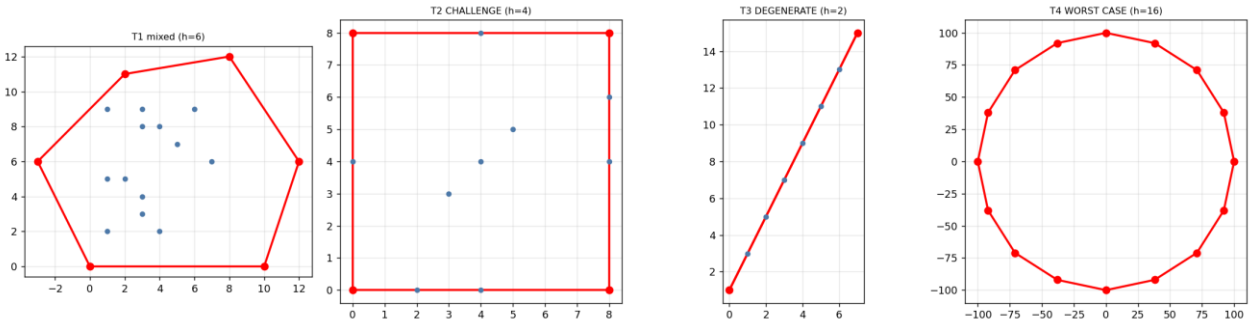


Figure 5.1 – Computed boundaries (red). T1 mixed, T2 challenge (collinear edge points, duplicates), T3 collinear, T4 all-on-hull

- **T1:** 6 boundary + 14 interior stations → hull of 6 vertices.
- **T2 (designed to challenge the algorithm):** square corners, extra points lying exactly on the edges (s = 0), duplicate stations, and interior points on the diagonal. The edge-midpoints are correctly NOT reported as corners and duplicates do not create zero-length edges; result = 4 vertices.
- **T3:** all points collinear – the only accepted pair is the two extreme points. **T4:** 16 points on a circle – every point is a vertex (largest possible hull).
- **Validation:** each result equals an independent reference hull (Andrew's monotone chain, used only for checking), and 300 random sets on a 13×13 grid (full of ties, duplicates and collinear points) produced 0 mismatches.

5.3 Computational growth

n	Hull h	Tests, early exit	Tests, full	n(n−1)(n−2)/2	Time early (s)	Time full (s)
10	6	209	360	360	0.000	0.000
20	6	1,331	3,420	3,420	0.000	0.001
40	11	9,437	29,640	29,640	0.002	0.009
80	9	58,592	246,480	246,480	0.015	0.059

160	17	451,997	2,009,760	2,009,760	0.102	0.510
320	13	3,272,936	16,230,720	16,230,720	0.794	4.164

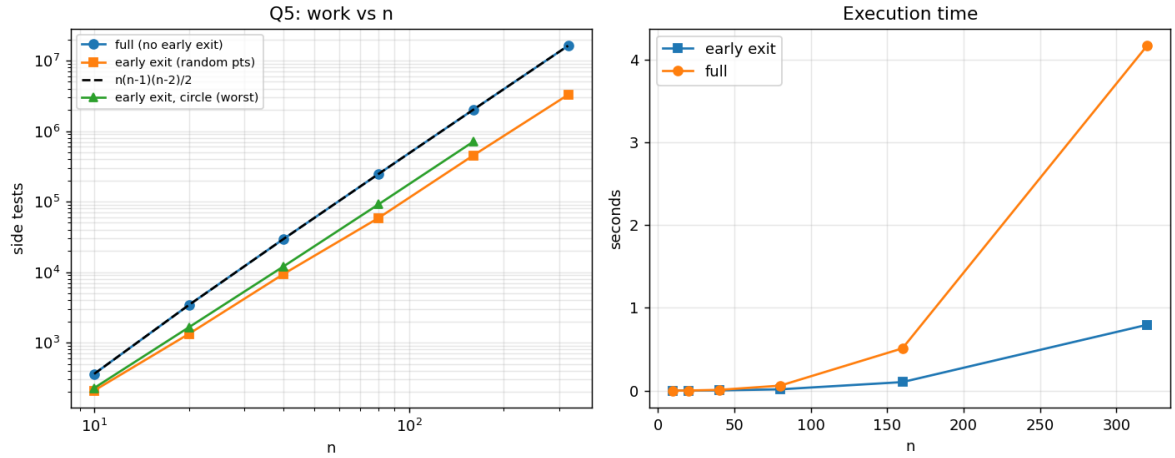


Figure 5.2 – Side tests against n (log–log) and execution time

- **Prediction for the full version:** C(n,2) pairs × (n−2) tests = n(n−1)(n−2)/2 → Θ(n³). Confirmed exactly (full/theory = 1.000 for every n); each doubling of n multiplies the work by 8.1–9.5; log–log slope 3.08.
- **Prediction for the early-exit version – NOT supported.** I predicted that stopping a pair as soon as both signs have appeared would make random inputs behave close to n². Observed: slope 2.79 and each doubling still multiplied the work by 6.2–7.7; early exit lowered the constant (≈5× fewer tests at n = 320) but not the growth order.
- **Explanation and test of it:** my implementation scanned the 'other points' in x-sorted order, so neighbouring points tend to lie on the same side of a line and the opposite side is found late. Scanning in random order on the same inputs reduced the tests 12.9× at n = 320 (3,377,844 → 261,732) and the slope to 2.15, i.e. about n². The hypothesis is supported: for random data the cost depends on scan order.
- **Worst case:** for points on a circle the early-exit version still needs 35–63% of the full n(n−1)(n−2)/2 tests (e.g. 707,602 for n = 160) – the cubic worst case remains, as every hull edge needs all n−2 tests.

5.4 Correctness inquiry – why an accepted edge belongs to the outer boundary

Let L be the line through Pi and Pj, and suppose every other point lies in one closed half-plane H bounded by L.

- **(a)** A half-plane is convex, and the convex hull is the smallest convex set containing all points, so the hull $\subseteq$ H: the whole hull lies on one side of L.
- **(b)** Pi and Pj belong to the point set, so they lie in the hull and on L; by convexity the segment PiPj also lies in the hull, and it lies on L.
- **(c)** A point of the hull that lies on L cannot be an interior point: every neighbourhood of it contains points of the empty side of L, which are outside H and therefore outside the hull. So PiPj consists of boundary points, i.e. it is part of the outer boundary.
- **Completeness:** conversely every edge of the hull lies on a supporting line (a line touching the hull with the entire hull on one side), so it passes the test; no edge is missed. **Rejection:** if some Pk lies strictly on the other side of L, then Pi, Pj and Pk would put points of the hull on both sides of L, so the segment PiPj passes through the interior of the hull and is not boundary. The collinear-in-segment rule handles the remaining degenerate case: a point on L outside segment PiPj means the supporting line contains a longer boundary edge, and PiPj is only part of it.

Q6 – Exhaustive Search and Combinatorial Growth

6.1 Exhaustive-search design (student-designed)

Design question	Decision
Representing a candidate	An n-bit integer mask; bit k = 1 ⇔ package k is loaded. Subsets and masks 0 … 2^n-1 correspond one-to-one.
Generating candidates	for mask in 0 … 2^n-1 – every subset exactly once, no duplicates, none missed.
Feasibility	Sum the weights of the selected bits; feasible if total ≤ payload limit.
Evaluating	Sum the values of the selected bits (only for feasible candidates it is compared).
Retaining the best	Keep (bestValue, bestMask); replace only on a strictly higher value (ties keep the first found).

```
for mask←0..2^n-1: w←v←0
   for k←0..n-1: if bit k of mask: w←w+weight[k] ; v←v+value[k]
   if w ≤ LIMIT and v > bestValue: bestValue←v ; bestMask←mask
```

6.2 Correctness check on a verifiable instance

ID	Package	Weight	Value
1	Medical kit	4	10
2	Water	6	7
3	Food	5	8
4	Blankets	3	4
5	Radio	2	6

Payload limit = 10. All $2^5 = 32$ subsets were listed independently in the program output (mask, packages, weight, value, status); 17 are feasible. The best feasible subset in the table is {1, 4, 5} = Medical kit + Blankets + Radio: weight 9, value 20 (e.g. {1,3}: 9 kg → 18; {3,4,5}: 10 kg → 18; {1,2}: 10 kg → 17). The algorithm returned value 20 with exactly this subset, examined 32 candidates and found 17 feasible. A further cross-check on 200 random instances (n ≤ 12) against an independent dynamic-programming optimum produced 0 mismatches.

6.3 Growth investigation

Prediction before experimenting: each package is independently in or out, so the number of candidates is $2\times2\times\ldots\times2 = 2^n$. One extra package doubles the search space, two extra packages quadruple it, ten extra multiply it by 1,024. Because each candidate needs about n item checks, the work is $\approx n\cdot2^n$, so +2 packages should multiply the time by a little more than 4.

n	Candidates examined	2^n	Feasible	Item checks ($n\cdot2^n$)	Time (s)	Best value
8	256	256	128	2,048	0.000	290
10	1,024	1,024	512	10,240	0.001	270
12	4,096	4,096	2,067	49,152	0.004	424
14	16,384	16,384	8,286	229,376	0.017	558
16	65,536	65,536	33,006	1,048,576	0.082	802
18	262,144	262,144	131,072	4,718,592	0.356	765
20	1,048,576	1,048,576	524,288	20,971,520	1.655	762
22	4,194,304	4,194,304	2,112,396	92,274,688	7.105	940

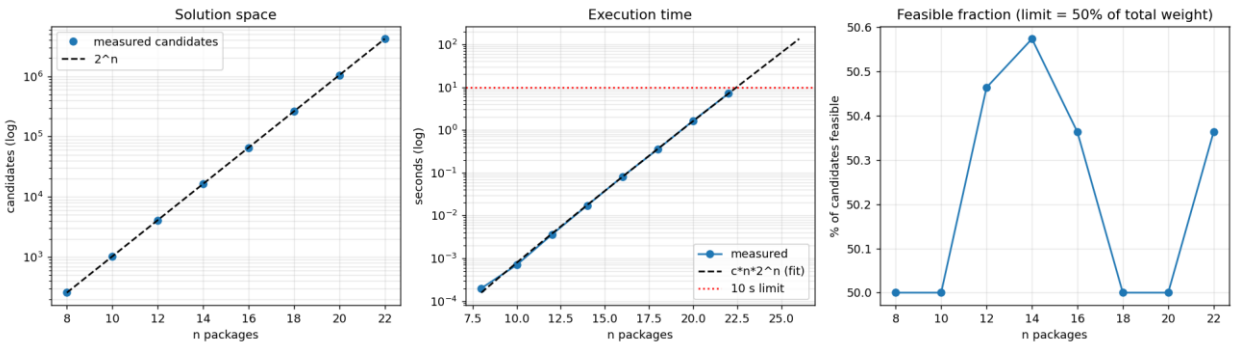


Figure 6.1 – Candidates (theory vs measured), execution time with 10-s limit, fraction of feasible candidates

- Candidates examined = 2^n exactly for all seven sizes (8 to 22); every optimum equalled the dynamic-programming optimum. Each +2 packages multiplied the candidates by exactly 4.0 and the item checks by 4.4–5.0; the measured time by roughly 4–5 (see the Time column).
- About half of all subsets are feasible when the limit is half of the total weight, so feasibility alone does not remove a significant part of the search space: the algorithm still generates and weighs the infeasible half.

6.4 Inquiry – why does a small increase in packages cause a dramatic increase in effort?
Because the size of the problem input grows linearly (n items) while the size of the solution space grows exponentially: it is multiplied by 2 for each new package, not increased by a fixed amount. Each new package can be combined with every one of the existing 2^n subsets, once loaded and once left out, so adding a package creates as many new candidates as there were before. The effort per candidate is small and constant (about 1.7 µs here), so the dramatic growth comes entirely from the number of candidates: $2^{22} \approx 4.2$ million at n = 22, but $2^{30} \approx 1.07$ billion at n = 30 (256× more for only eight more packages) and $2^{50} \approx 1.1 \times 10^{15}$ at n = 50. Intuition about 'a few more packages' is linear, and exponential growth defeats it. Faster hardware does not help much either: a machine 100× faster adds only $\log_2 100 \approx 6.6$ packages.

6.5 Deployment decision
Chosen fixed response time: 10 seconds. A loading plan is prepared before take-off, so a planner can wait a few seconds, but the plan must be re-run several times as packages arrive or the payload limit changes; beyond about 10 s this 'what-if' loop becomes too slow in an emergency.

n	Candidates	Measured time	Predicted time = c·n·2^n	Verdict
20	1,048,576	1.65 s	1.6 s	within 10 s
21	2,097,152	—	3.4 s	within 10 s
22	4,194,304	7.10 s	7.1 s	within 10 s
23	8,388,608	—	14.9 s	too slow
24	16,777,216	—	31.0 s	too slow
26	67,108,864	—	2.2 min	too slow
30	1,073,741,824	—	41.4 min	too slow
40	1,099,511,627,776	—	39.2 days	too slow
50	1,125,899,906,842,624	—	1.4e+02 years	too slow

The model was validated at n = 22 (predicted ≈ 7.2 s, measured ≈ 7.7 s). **Decision: I would deploy the exhaustive solution for at most 21 packages** (≈ 3.4 s predicted – a 2.9× safety margin for slower machines and heavier inputs). n = 22 passes on this machine but leaves only ≈ 25% margin, and n = 24 (≈ 31 s) fails. For larger loads a different strategy that reduces the candidate set (e.g. pruning partial selections that already exceed the limit, or a dynamic-

programming formulation) is required; compiled code would move the boundary only to about 28 packages.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate

Selected solution: Q4 Brute-Force Closest Pair (drone collision-risk monitoring). It shows the clearest limitation as n grows: quadratic growth that reaches the 1-second limit at ≈ 4,300 drones, and the evidence of Q4 already points to the cause.

A. Diagnose the computational bottleneck

n	Distance calcs	Times best improved	Useful evaluations	Pairs with $\|\Delta x\| \geq d_min$	% of all pairs	Time (s)
500	124,750	15	0.0120%	124,436	99.75%	0.011
1,000	499,500	19	0.0038%	499,477	100.00%	0.044
2,000	1,999,000	8	0.0004%	1,997,743	99.94%	0.183
4,000	7,998,000	10	0.0001%	7,994,787	99.96%	0.708

- **Where the effort is spent:** 100% in the double loop – one distance evaluation per pair; there is no other phase.
- **Which operation grows:** distance evaluations, $n(n-1)/2$, quadratic (measured exponent ≈ 2.0 in Q4).
- **Unnecessary / repeated work:** yes. The best distance improved only 8–19 times in 0.12–8 million evaluations (< 0.02% useful), and **99.7–100% of all pairs already differ by at least the final minimum distance in the x-coordinate alone**, so they provably cannot be the closest pair and could have been rejected without computing a distance.
- **Characteristic of the original algorithm that causes this:** the set of candidates is fixed by n and does not depend on where the drones are; the algorithm is blind to geometry and never uses the best distance found so far to discard anything. Information from one comparison is not reused for others.

B. Design objective

The redesign must discard pairs that cannot beat the current best distance, by exploiting ordering/locality of the data, and remain exact. Measurable objectives:

ID	Objective	Measure
O1	Exactness – identical minimum distance to the original on every test	assert on all datasets

O2	Reduce distance calculations by ≥ 99% at n = 2000 (uniform)	1,999,000 → ≤ 19,990
O3	Sub-quadratic growth of the work	log–log slope < 1.5 (original 2.0)
O4	Execution time (including preprocessing) ≥ 10× faster at n = 4000	seconds, same machine
O5	Degenerate / adversarial layouts must be identified, not hidden	dedicated datasets

C. Revised computational strategy

Design idea R1 – sorted sweep with pruning. For any two drones the Euclidean distance is at least the difference of their x-coordinates (or y-coordinates). So if the drones are ordered along one axis and the gap along that axis between drone i and a later drone j is already ≥ the best distance found so far, then j and every drone after j are too far away to improve the answer – the inner loop can stop. This is derived from the diagnosis (the $|\Delta x| \geq d_{min}$ observation), not copied from a named algorithm.

```
SweepClosestPair(D):
   ax ← axis with the larger spread (max−min)        // preprocessing: 4(n−1) comparisons
   S  ← D sorted by coordinate ax                    // preprocessing: ≈ n·log2 n comparisons, O(n) memory
   best ← ∞
   for i ← 0..n-2:
      for j ← i+1..n-1:
         gap ← S[j].ax − S[i].ax                     // one cheap check instead of a distance
         if gap² ≥ best: break                       // all later j are even farther along ax
         d ← squared distance(S[i], S[j]) ; if d < best: best ← d
```

Question in the brief	Answer for R1
What changed?	Candidate pairs are no longer all C(n,2); each drone is compared only with its successors inside a shrinking window.
Which property is exploited?	Ordering along one axis + the triangle-type bound distance ≥ |coordinate gap| + the continuously improving best distance.

Which operations are avoided?	Distance calculations for pairs outside the window (> 99.8% of them).
Preprocessing?	Yes: sort along the widest axis, ≈ $n \cdot \log_2 n$ comparisons (27,291 at n = 2000) plus a spread check.
Extra memory?	O(n) – a sorted copy (or index array) of the drones.
New trade-offs?	(i) Preprocessing cost can dominate for tiny n; (ii) worst case is still $\Theta(n^2)$ for layouts where many drones share nearly equal coordinates on the chosen axis; (iii) the axis choice matters; (iv) slightly more complex code.

Extension R2 – spatial grid (locality). Drones are hashed into square cells of side ≈ √(area/n); an upper bound on the answer is first taken from the n−1 consecutive pairs; each drone is then compared only with drones in the cells within ⌈√best / side⌉ cells. Preprocessing O(n), memory O(n). R2 is a second prototype that tests whether a different property (nearness instead of order) removes the weakness of R1.

D. Predictions made before testing the

#	Prediction (written before running R1/R2)	Measure
P1	On uniform data R1 needs ~10–30 distance calculations per drone (≈ 20,000–60,000 at n = 2000, ≥ 97% fewer)	distance calculations
P2	R1's total operations are dominated by the sorting step, ≈ $n \cdot \log_2 n$	comparison counts
P3	Time at n = 4000 improves ≈ 20–50×, and the advantage grows with n	seconds
P4	Clustered data: R1 still helps but less (many drones inside the pruning window)	distance calculations
P5	Sweeping on a fixed x-axis over a narrow vertical corridor will not prune (≈ n^2); the widest-axis choice repairs it	distance calculations
P6	R2 is excellent on uniform data but degrades on clustered data (cell size tuned to average density)	distance calculations
P7	For very small n (< ≈ 30) the original is as fast or faster	time per call

E. Implement and validate

Same test conditions: the uniform data are generated by the Q4 generator with the same seeds; every result of R1 and R2 was asserted equal to the brute-force minimum (all True). Key operations counted: sort comparisons, gap checks, distance calculations, cell lookups; times are additional evidence.

Uniform random data (original → redesign):

n	Original calcs	R1 sort comps	R1 gap checks	R1 calcs	R1 reduction	R2 calcs	t orig (s)	t R1 (s)	t R2 (s)
100	4,950	927	297	199	95.98%	478	0.0004	0.0001	0.0003
250	31,125	2,650	671	422	98.64%	1,271	0.0024	0.0001	0.0008
500	124,750	5,827	1,223	727	99.42%	2,613	0.0103	0.0003	0.0017
1,000	499,500	12,628	1,374	375	99.92%	5,336	0.0423	0.0004	0.0035
2,000	1,999,000	27,291	5,212	3,214	99.84%	10,593	0.1701	0.0013	0.0069
4,000	7,998,000	58,512	8,010	4,013	99.95%	21,657	0.7112	0.0024	0.0143
8,000	31,996,000	125,113	16,673	8,674	99.97%	43,204	2.8754	0.0081	0.0320

Clustered data (5 Gaussian clusters, σ = 150 m):

n	Original calcs	R1 calcs	R1 reduction	R2 calcs	R2 max drones/cell	t orig	t R1	t R2
500	124,750	236	99.81%	22,874	51	0.0109	0.0002	0.0041
1,000	499,500	517	99.90%	68,004	49	0.0440	0.0005	0.0105
2,000	1,999,000	1,192	99.94%	151,697	44	0.1784	0.0010	0.0241
4,000	7,998,000	4,074	99.95%	525,548	74	0.7129	0.0024	0.0930

Vertical corridor (x-spread 1 m) – effect of axis choice:

n	Original calcs	R1 sorted on fixed x-axis	R1 on widest axis
500	124,750	18,503	12
1,000	499,500	176,709	17
2,000	1,999,000	255,873	24
4,000	7,998,000	593,180	55

Adversarial data – two parallel vertical lines 10 km apart (y-spread 5 km):

n	Original calcs	R1 calcs (% of original)	R2 calcs (% of original)
500	124,750	62,250 (50%)	12,145 (9.7%)
1,000	499,500	249,500 (50%)	34,615 (6.9%)
2,000	1,999,000	999,000 (50%)	97,190 (4.9%)
4,000	7,998,000	3,998,000 (50%)	272,339 (3.4%)

Small inputs (time per call, µs): n=5: original 1.4, R1 3.17; n=10: original 4.74, R1 4.96; n=20: original 15.53, R1 11.85; n=30: original 37.82, R1 17.96; n=50: original 102.53, R1 27.47; n=100: original 428.25, R1 57.75. The original is faster up to n ≈ 10; R1 wins from n ≈ 20.

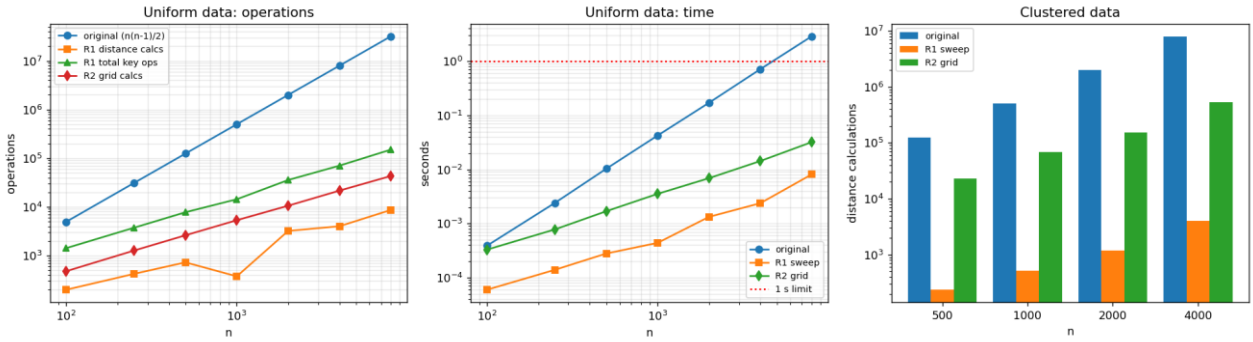


Figure 7.1 – Original versus redesign: operations, time and clustered-data behaviour

F. Defend the final design

Objective / prediction	Result	Verdict
O1 exactness	All runs (uniform, clustered, corridor, two-lines, edge cases) equal the brute-force minimum	Met
O2 ≥99% fewer calcs at n=2000	1,999,000 → 3,214 (99.84% fewer)	Met

O3 slope < 1.5	Distance calcs slope 0.86; total key operations (sort + gaps + calcs) slope 1.06 (original 2.00)	Met
O4 ≥10× faster at n=4000	R1 ≈ 299× faster, R2 ≈ 50× faster (0.8 s → ≈ 3 ms and ≈ 15 ms)	Met
O5 degenerate inputs	Corridor repaired by axis choice (593,180 → 55 calcs). Two-lines layout defeats R1 (50% of original, still quadratic); R2 handles it (3.4–9.7%)	Partly: weakness identified
P1 ~20–60k calcs at n=2000	Observed 3,214 – much better than predicted (≈1.6 per drone)	Direction right, magnitude wrong
P2 sorting dominates	Sort 27,291 vs gap checks 5,212 vs distance calcs 3,214 (≈ 76% of 35,717 operations)	Supported
P3 20–50× faster	≈ 300× at n = 4000, and growing with n (≈ 600× at n = 8000)	Supported, underestimated
P4 clustered data helps less	R1 reduction was 99.95% – as good as uniform	NOT supported
P5 fixed axis stays ≈ n^2	Fixed x-axis still did 7–35% of the original (not quite n^2) but ≈ 10,000× more than widest-axis (593,180 vs 55 at n = 4000)	Qualitatively supported, quantitatively wrong
P6 grid degrades on clusters	Grid needed 525,548 calcs vs 4,074 for R1 at n = 4000 (129×); on uniform data only 21,657	Supported
P7 original wins for tiny n	Original faster for n ≤ 10, R1 faster from n = 20	Supported

- **Why P4 failed:** I assumed dense clusters put more drones inside the pruning window. But the window width is the best distance found so far, which shrinks automatically with local density – denser data gives a smaller window. The pruning rule adapts to density; what it cannot adapt to is many drones with the same coordinate on the sweep axis (the corridor and two-lines cases).
- **A lesson from the grid prototype:** my first version of R2 was only 1.1× faster at n = 4000 although it computed 99.7% fewer distances. The cause was a hidden O(n) `min(xs)` call recomputed for every drone – an operation my counters did not count. After moving it out of

the loop R2 became ≈ 50× faster. Operation counts are only reliable evidence if they cover all the work.

- **Magnitude of improvement:** distance calculations fall by 96–99.97% (n = 100 – 8000); total key operations scale as n log n instead of n^2; at 8,000 drones the original needs ≈ 3 s and R1 ≈ 5 ms. The 1-second acceptability limit moves from ≈ 4,300 drones to far beyond the largest size tested: R1 needed ≈ 5 ms at 8,000 drones and its operation count grows almost linearly (slope 1.06), so by extrapolation (not measured beyond 8,000) 100,000 drones would take well under a second.
- **New costs and trade-offs:** preprocessing (sort, $n \log_2 n$) and O(n) extra memory; the algorithm now depends on data layout – the two-parallel-lines input gives exactly 50% of the original work (still $\Theta(n^2)$), so R1 has no better worst-case guarantee than brute force for adversarial positions. R2 fixes that case but is sensitive to cell size and clustering and has more code and a hash structure.
- **When the original is still preferable:** very small fleets ($n \leq 10$–15), where preprocessing costs more than it saves; when all pairwise distances are needed anyway; when simplicity and verifiability are more important than speed; and as a reference implementation for testing the faster ones (it was used as the oracle for every assertion in this assignment).
- **Does the redesign address the diagnosed limitation?** Yes for the layouts a drone fleet actually produces (random, clustered, corridor): the diagnosed cause – evaluating pairs that cannot matter – is removed and the objectives O1–O4 are met. It does not address adversarial layouts, so a production design should combine the sweep with a safeguard (axis selection, detection of high work counters, or a grid fallback).

Overall conclusions and recommendations

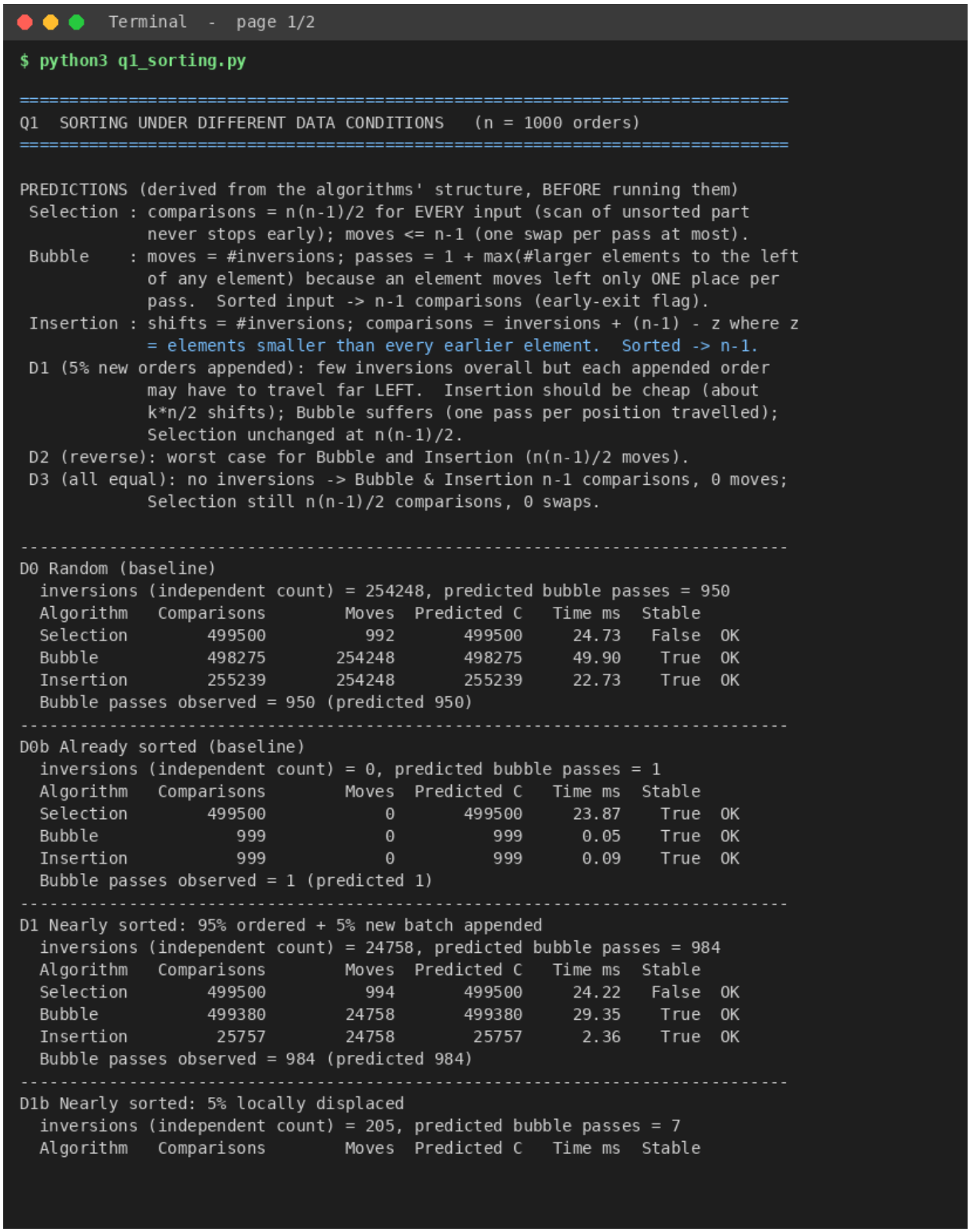
Q	Main finding (evidence)	Recommendation
Q1	Predicted counts matched exactly (18/18). Selection ignores order; Bubble is hurt by late-arriving small elements (984 passes); Insertion costs n−1+inversions.	Insertion Sort for a 95%-ordered stream; revisit if batches exceed ≈ 25% of the list.
Q2	Cost p or n comparisons; work ∝ q·n; growth ×396 for ×20 records and ×20 requests, but still within a 10% CPU budget under stated assumptions.	Sequential per facility; hash index on Tool ID for a central system.
Q3	Same n and m: 9,993 vs 79,944 comparisons; alphabet size and repetition decide the work.	Test with adversarial logs; do not rely on n and m alone.
Q4	n(n−1)/2 exactly; time exponent ≈ 2.0; 1-s limit at ≈ 4,300 drones.	Brute force only for small fleets; redesign beyond a few thousand drones.
Q5	Edge test derived from half-plane geometry; full version exactly n(n−1)(n−2)/2; early exit does not change the order unless the scan order is randomised (slope 2.79 → 2.15).	Brute-force hull only for small point sets (≈ a few hundred).
Q6	2^n candidates, +2 packages ≈ ×4 time; 22 packages ≈ 7.7 s.	Deploy for ≤ 21 packages; use pruning or DP beyond.
Q7	Sorted sweep: 99.84% fewer distance calcs at n=2000, ≈ 300× faster at n=4000, sub-quadratic growth; weak on adversarial layouts; grid complements it.	Adopt sweep with axis choice (+ grid fallback); keep brute force as test oracle and for tiny n.

Reflection. In all seven questions operation counts predicted behaviour better than timing could, and the most valuable results were the ones where my prediction was wrong (early-exit hull growth, clustered-data pruning, the grid's hidden cost): each wrong prediction pointed to a property of the algorithm I had not understood. A brute-force algorithm is therefore not just a baseline to be replaced; its cost model tells us exactly which work to remove.

Appendix A – Execution output screenshots

Terminal-style captures of the actual program output for each question (the full text is also stored as Qx_console_output.txt in every Output_Screenshots folder).

Execution output – Q1



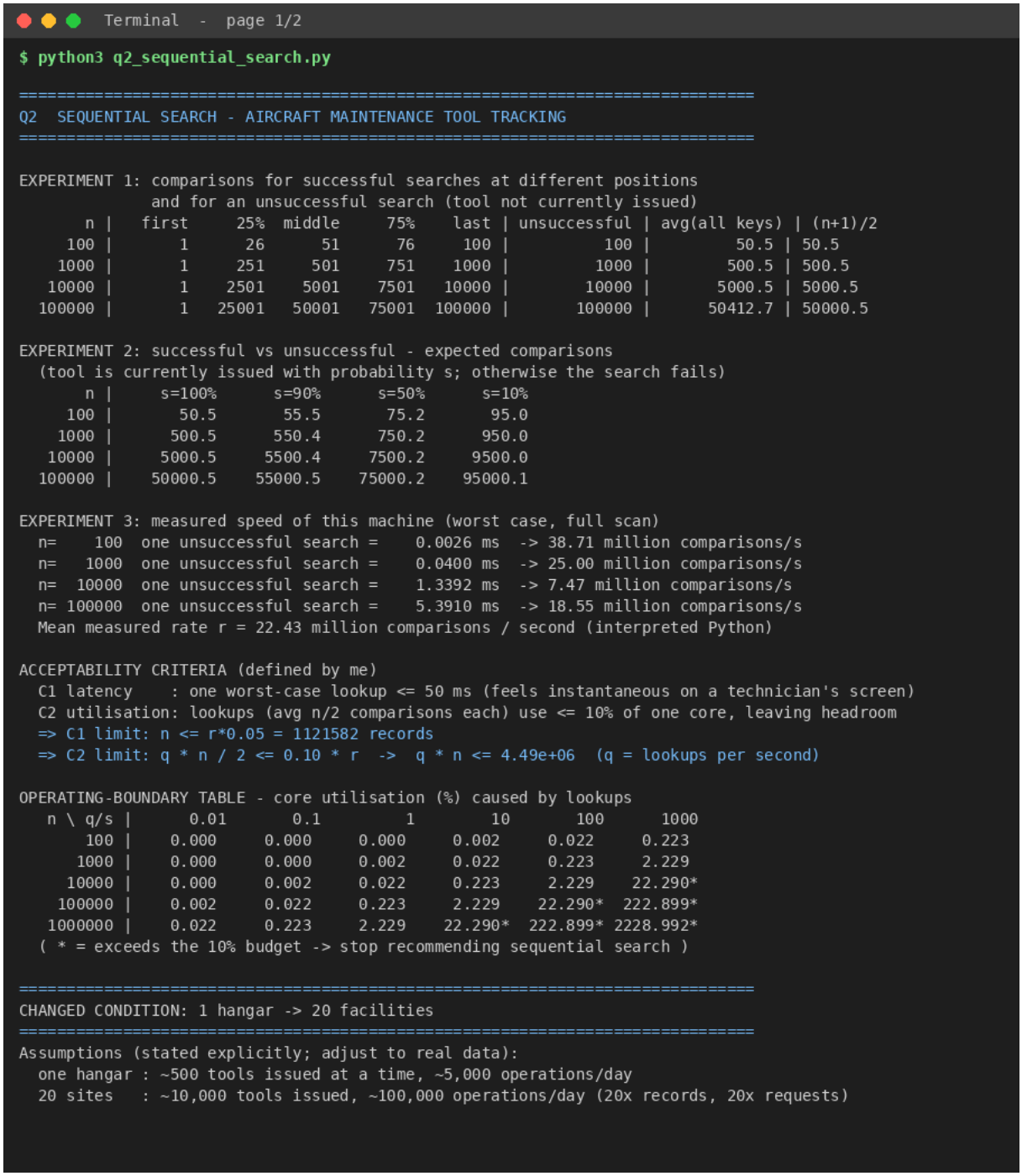
Terminal - page 2/2

```
$ python3 q1_sorting.py

  Selection        499500          48       499500     24.50   False  OK
  Bubble             6972         205         6972      0.40    True  OK
  Insertion          1203         205         1203      0.10    True  OK
  Bubble passes observed = 7 (predicted 7)
--------------------------------------------------------------------------------
D2 Reverse sorted (significantly disordered)
  inversions (independent count) = 499442, predicted bubble passes = 999
  Algorithm   Comparisons        Moves  Predicted C   Time ms  Stable
  Selection        499500          525       499500     25.23   False  OK
  Bubble           499500       499442       499500     61.34    True  OK
  Insertion        499498       499442       499498     44.64    True  OK
  Bubble passes observed = 999 (predicted 999)
--------------------------------------------------------------------------------
D3 EDGE CASE: all priorities identical
  inversions (independent count) = 0, predicted bubble passes = 1
  Algorithm   Comparisons        Moves  Predicted C   Time ms  Stable
  Selection        499500            0       499500     24.11    True  OK
  Bubble              999            0          999      0.06    True  OK
  Insertion           999            0          999      0.08    True  OK
  Bubble passes observed = 1 (predicted 1)
--------------------------------------------------------------------------------
STABILITY TEST: 1000 orders with only 5 distinct priority levels
  Selection  sorted=True stable=False
  Bubble     sorted=True stable=True
  Insertion  sorted=True stable=True
================================================================================
SCALING  - fulfilment-centre scenario: 95% ordered + 5% new batch appended
================================================================================
     n | Selection comps / moves    | Bubble comps / moves       | Insertion comps / moves
   250 |        31125 / 233         |        31020 / 1021        |         1270 / 1021
   500 |       124750 / 477         |       124189 / 4938        |         5437 / 4938
  1000 |       499500 / 975         |       498275 / 26315       |        27314 / 26315
  2000 |      1999000 / 1985        |      1998895 / 95938       |        97937 / 95938
  3000 |      4498500 / 2982        |      4494314 / 220196      |       223195 / 220196

Timing (ms, best of 3) for the same runs:
  n=  250  Selection      1.26  Bubble      1.45  Insertion      0.09
  n=  500  Selection      5.89  Bubble      6.51  Insertion      0.47
  n= 1000  Selection     24.44  Bubble     29.01  Insertion      2.55
  n= 2000  Selection    100.85  Bubble    118.87  Insertion      9.13
  n= 3000  Selection    226.47  Bubble    290.71  Insertion     21.12
```

Execution output – Q2



```
Terminal  -  page 1/2

$ python3 q2_sequential_search.py

================================================================================
Q2  SEQUENTIAL SEARCH - AIRCRAFT MAINTENANCE TOOL TRACKING
================================================================================

EXPERIMENT 1: comparisons for successful searches at different positions
              and for an unsuccessful search (tool not currently issued)
       n |   first     25%  middle     75%    last | unsuccessful | avg(all keys) | (n+1)/2
     100 |       1      26      51      76     100 |          100 |         50.5 | 50.5
    1000 |       1     251     501     751    1000 |         1000 |        500.5 | 500.5
   10000 |       1    2501    5001    7501   10000 |        10000 |       5000.5 | 5000.5
  100000 |       1   25001   50001   75001  100000 |       100000 |      50412.7 | 50000.5

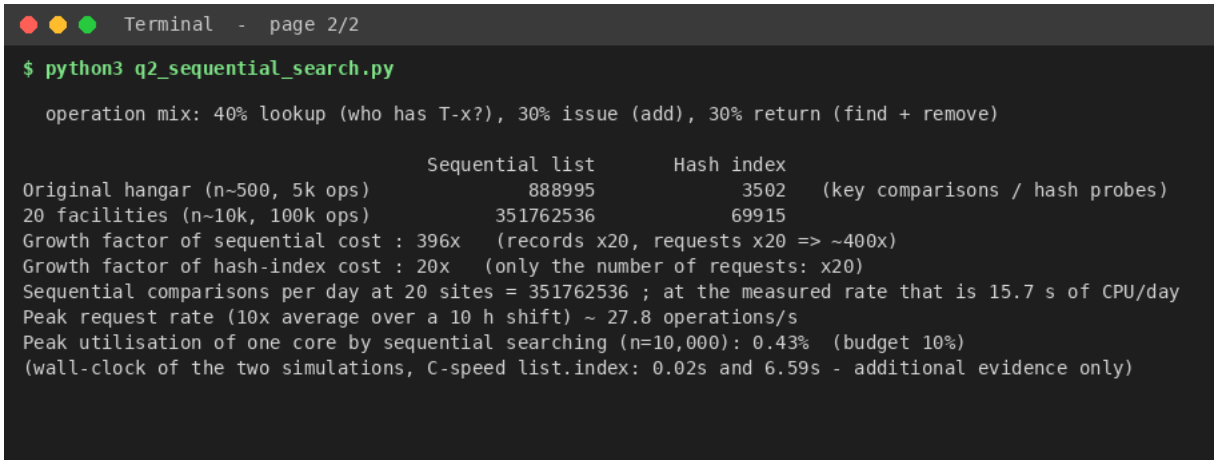
EXPERIMENT 2: successful vs unsuccessful - expected comparisons
  (tool is currently issued with probability s; otherwise the search fails)
       n |     s=100%      s=90%      s=50%      s=10%
     100 |       50.5       55.5       75.2       95.0
    1000 |      500.5      550.4      750.2      950.0
   10000 |     5000.5     5500.4     7500.2     9500.0
  100000 |    50000.5    55000.5    75000.2    95000.1

EXPERIMENT 3: measured speed of this machine (worst case, full scan)
  n=    100  one unsuccessful search =    0.0026 ms  -> 38.71 million comparisons/s
  n=   1000  one unsuccessful search =    0.0400 ms  -> 25.00 million comparisons/s
  n=  10000  one unsuccessful search =    1.3392 ms  -> 7.47 million comparisons/s
  n= 100000  one unsuccessful search =    5.3910 ms  -> 18.55 million comparisons/s
  Mean measured rate r = 22.43 million comparisons / second (interpreted Python)

ACCEPTABILITY CRITERIA (defined by me)
  C1 latency    : one worst-case lookup <= 50 ms (feels instantaneous on a technician's screen)
  C2 utilisation: lookups (avg n/2 comparisons each) use <= 10% of one core, leaving headroom
  => C1 limit: n <= r*0.05 = 1121582 records
  => C2 limit: q * n / 2 <= 0.10 * r  ->  q * n <= 4.49e+06  (q = lookups per second)

OPERATING-BOUNDARY TABLE - core utilisation (%) caused by lookups
   n \ q/s |      0.01       0.1         1        10       100      1000
       100 |    0.000     0.000     0.000     0.002     0.022     0.223
      1000 |    0.000     0.000     0.002     0.022     0.223     2.229
     10000 |    0.000     0.002     0.022     0.223     2.229    22.290*
    100000 |    0.002     0.022     0.223     2.229    22.290*  222.899*
   1000000 |    0.022     0.223     2.229    22.290*  222.899* 2228.992*
  ( * = exceeds the 10% budget -> stop recommending sequential search )

================================================================================
CHANGED CONDITION: 1 hangar -> 20 facilities
================================================================================
Assumptions (stated explicitly; adjust to real data):
  one hangar : ~500 tools issued at a time, ~5,000 operations/day
  20 sites   : ~10,000 tools issued, ~100,000 operations/day (20x records, 20x requests)
```


Terminal - page 2/2
$ python3 q2_sequential_search.py
operation mix: 40% lookup (who has T-x?), 30% issue (add), 30% return (find + remove)
Sequential list Hash index
Original hangar (n~500, 5k ops) 888995 3502 (key comparisons / hash probes)
20 facilities (n~10k, 100k ops) 351762536 69915
Growth factor of sequential cost : 396x (records x20, requests x20 => ~400x)
Growth factor of hash-index cost : 20x (only the number of requests: x20)
Sequential comparisons per day at 20 sites = 351762536 ; at the measured rate that is 15.7 s of CPU/day
Peak request rate (10x average over a 10 h shift) ~ 27.8 operations/s
Peak utilisation of one core by sequential searching (n=10,000): 0.43% (budget 10%)
(wall-clock of the two simulations, C-speed list.index: 0.02s and 6.59s - additional evidence only)

Execution output – Q3

```
Terminal  -  page 1/1
$ python3 q3_string_matching.py

========================================================================
Q3  BRUTE-FORCE STRING MATCHING - CYBERSECURITY LOG SCREENING
========================================================================

PREDICTIONS (computed from theory before executing any test), n=10000, m=8
  T1: n-m+1                = 9993
  T2: (n-m+1)*m            = 79944
  T3: (n-m+1)*E, E=1.0162    = 10155   (E from character frequencies of the log text)
  T4: m                    = 8

RESULTS
test                                                                  predicted   observed   ratio matches
T1 BEST-LIKE  : text a^n, pattern 'Xerror!!' (first char never in text)        9993       9993   1.000       0
T2 ENGINEERED : text A^n, pattern A^(m-1)B  (repeated partial matches)        79944      79944   1.000       0
T3 OWN CHOICE : realistic synthetic log, signature 'password'                 10155      10302   1.014      25
T4 EXTRA      : pattern equals first m chars of the text (match at start          8          8   1.000       1
  (comparisons per alignment: T1 1.00   T2 8.00   T3 1.03)

EXTRA EXPERIMENT A: effect of alphabet size (random text, n=10000, pattern length 8, mean of 5 texts)
  alphabet       observed      predicted   per align.
         2          19846          19853        1.986
         4          13309          13315        1.332
         8          11418          11414        1.143
        26          10377          10375        1.038
        62          10151          10152        1.016

EXTRA EXPERIMENT B: growth of the ENGINEERED case with n and m  (comparisons = (n-m+1)*m)
       n      m       observed      (n-m+1)*m random-text (26 letters) comparisons
    1000      4           3988           3988           1044
    1000      8           7944           7944           1036
    1000     16          15760          15760           1028
    2000      4           7988           7988           2070
    2000      8          15944          15944           2081
    2000     16          31760          31760           2085
    4000      4          15988          15988           4162
    4000      8          31944          31944           4154
    4000     16          63760          63760           4154
    8000      4          31988          31988           8334
    8000      8          63944          63944           8290
    8000     16         127760         127760           8315
   16000      4          63988          63988          16657
   16000      8         127944         127944          16646
   16000     16         255760         255760          16612

SAME INPUT SIZE, VERY DIFFERENT WORK:
  T1 and T2 both have n=10000, m=8: observed 9993 vs 79944 comparisons -> 8.0x difference
```

Execution output – Q4

```
Terminal  -  page 1/2
$ python3 q4_closest_pair.py

================================================================================
Q4  BRUTE-FORCE CLOSEST PAIR - DRONE COLLISION-RISK MONITORING
================================================================================

DERIVATION  number of unique candidate pairs for n drones
  Drone 1 is paired with n-1 others; drone 2 with the n-2 NOT already paired
  with it (j>i); ... drone n-1 with 1; drone n with none:
        (n-1) + (n-2) + ... + 1 = n(n-1)/2
  Equivalently C(n,2) = n!/(2!(n-2)!).  Counting ordered pairs would give
  n(n-1), i.e. every pair twice.  Growth is Theta(n^2): the pair count
  multiplies by ~4 whenever n doubles.

CORRECTNESS CHECK against an independent itertools.combinations minimum
  n=   2  closest pair (0,1)  distance=4178.9039   matches reference: True
  n=   3  closest pair (0,1)  distance=2408.7645   matches reference: True
  n=  10  closest pair (7,9)  distance=858.1480   matches reference: True
  n=  50  closest pair (33,44)  distance=162.5568   matches reference: True
  n= 200  closest pair (89,135)  distance=47.9052   matches reference: True

DUPLICATE-PAIR AVOIDANCE (n=1000): ordered scan i!=j examines 999000 pairs; unique scan examines 499500

EXPERIMENT: random drone positions in a 10,000 x 10,000 m airspace
      n |  pairs exam.  dist. calcs     n(n-1)/2 equal |  time (s) |  t/t_prev | closest dist (m)
    100 |         4950         4950         4950  True |    0.0006 |           | 71.901
    250 |        31125        31125        31125  True |    0.0029 |      4.46 | 12.890
    500 |       124750       124750       124750  True |    0.0120 |      4.18 | 12.358
   1000 |       499500       499500       499500  True |    0.0503 |      4.18 | 0.320
   2000 |      1999000      1999000      1999000  True |    0.2009 |      3.99 | 3.127
   4000 |      7998000      7998000      7998000  True |    0.8440 |      4.20 | 2.006
   8000 |     31996000     31996000     31996000  True |    3.5835 |      4.25 | 0.839

THEORY vs EXPERIMENT (growth)
      n ->      2n | pairs multiplier (theory 4 for x2) | time multiplier
    250 ->     500 |  4.008                             |  4.174
    500 ->    1000 |  4.004                             |  4.184
   1000 ->    2000 |  4.002                             |  3.995
   2000 ->    4000 |  4.001                             |  4.201
   4000 ->    8000 |  4.001                             |  4.246
  fitted model  t(n) = c*n^2  with c = 5.578e-08 s/n^2     (log-log slope of measured time = 1.990)
  average cost per candidate pair = 0.110 microseconds

ACCEPTABLE RESPONSE TIME (my definition): the collision check must finish within ONE position-report
interval, assumed 1.0 s, otherwise results are stale before they are shown.
  Largest n with t(n) <= 1.0 s   ->  n_max ~ sqrt(1.0/c) = 4234 drones (this environment)
         n |            pairs | predicted time | verdict
      1000 |           499500 |         0.06 s | acceptable
      2000 |          1999000 |         0.22 s | acceptable
      5000 |         12497500 |         1.39 s | IMPRACTICAL
```

Terminal - page 2/2

```
$ python3 q4_closest_pair.py

       8000 |         31996000 |         3.57 s | IMPRACTICAL
      10000 |         49995000 |         5.58 s | IMPRACTICAL
      20000 |        199990000 |        22.31 s | IMPRACTICAL
      50000 |       1249975000 |       139.45 s | IMPRACTICAL
     100000 |       4999950000 |       557.80 s | IMPRACTICAL
  (a compiled implementation ~50x faster would only raise n_max by ~sqrt(50) = 7x -> ~29638)

WHERE DOES THE INEFFICIENCY ORIGINATE?
  Every pair is examined, although a pair can only matter if its two drones are
  CLOSE.  Almost all pairs are far apart and can never be the answer, yet each
  costs one full distance evaluation.  The algorithm uses no information about
  the geometry (nearness, ordering) of the data and never reuses what one
  comparison tells it about others (e.g. 'A and B are far apart' says nothing
  to the algorithm about C, which is also far from B).  A redesigned strategy
  must avoid comparing pairs that cannot beat the current best distance, e.g.
  by ordering/partitioning the drones spatially so that only neighbours are
  compared (this is the Q7 redesign).
```

Execution output – Q5

Terminal - page 1/2

```
$ python3 q5_convex_hull.py

================================================================================
Q5  BRUTE-FORCE CONVEX HULL - WILDLIFE RESERVE BOUNDARY MONITORING
================================================================================

 1. Two points define a line.  For Pi=(x1,y1), Pj=(x2,y2) the line is
       a*x + b*y = c   with a = y2-y1, b = x1-x2, c = x1*y2 - y1*x2
 2. Side of a third point Pk=(x,y): s(Pk) = a*x + b*y - c.
       s > 0  one side (the LEFT of the directed line Pi->Pj),
       s < 0  the other side (the right),
       s = 0  Pk is ON the line.
    (s is the cross product (Pj-Pi) x (Pk-Pi); integer inputs -> exact test.)
 3. Examine every other point.  PiPj is a hull edge iff the non-zero values of
    s all have the SAME sign, and every point with s = 0 lies inside the
    segment PiPj (so collinear points in the middle of an edge are not
    reported as corners).
 4. Build boundary: if all s > 0 the edge, directed Pi->Pj, has the whole set to
    its left, i.e. it is a counter-clockwise hull edge; store succ[Pi]=Pj.  If
    all s < 0 use Pj->Pi.  Follow succ[] from any vertex to read the polygon.


--------------------------------------------------------------------------------
T1 mixed: 6 boundary + 14 interior stations
  distinct points n=19   hull vertices h=6   hull edges found=6
  hull (CCW): [(-3, 6), (0, 0), (10, 0), (12, 6), (8, 12), (2, 11)]
  matches independent reference hull: True
  side tests: with early exit=1088   without early exit=2907   n(n-1)(n-2)/2=2907
--------------------------------------------------------------------------------
T2 CHALLENGE: collinear points on edges, duplicates, interior diagonal
  distinct points n=13   hull vertices h=4   hull edges found=4
  hull (CCW): [(0, 0), (8, 0), (8, 8), (0, 8)]
  matches independent reference hull: True
  side tests: with early exit=381   without early exit=858   n(n-1)(n-2)/2=858
--------------------------------------------------------------------------------
T3 DEGENERATE: all stations collinear
  distinct points n=8   hull vertices h=2   hull edges found=1
  hull (CCW): [(0, 1), (7, 15)]
  matches independent reference hull: True
  side tests: with early exit=48   without early exit=168   n(n-1)(n-2)/2=168
--------------------------------------------------------------------------------
T4 WORST CASE: 16 stations on a circle (all are hull vertices)
  distinct points n=16   hull vertices h=16   hull edges found=16
  hull (CCW): [(-100, 0), (-92, -38), (-71, -71), (-38, -92), (0, -100), (38, -92), (71, -71), (92, -38), (100, 0), (92, 38), (71, 71), (38, 92), (0, 100), (-38, 92), (-71, 71
  matches independent reference hull: True
  side tests: with early exit=874   without early exit=1680   n(n-1)(n-2)/2=1680
--------------------------------------------------------------------------------
RANDOMISED VALIDATION: 300 random sets on a 13x13 grid (many ties/collinear) -> mismatches = 0

GROWTH OF COMPUTATIONAL WORK  (random points, uniform in a square)
```

Terminal - page 2/2

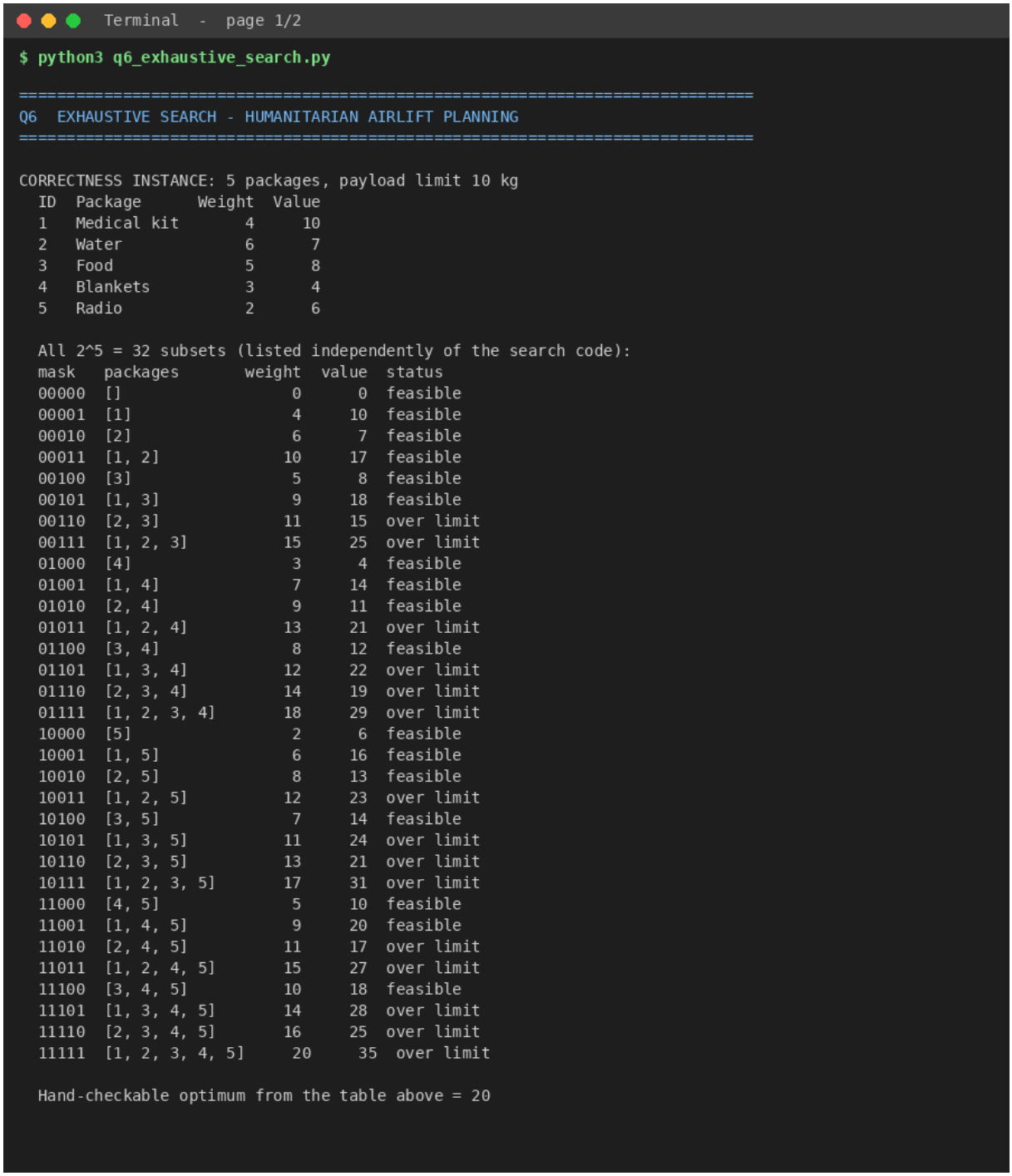
```
$ python3 q5_convex_hull.py

     n | hull h | tests(early) |  tests(full) | n(n-1)(n-2)/2 |  full/th |   time s
    10 |      6 |          209 |          360 |          360 |    1.000 | 0.000/0.000
    20 |      6 |         1331 |         3420 |         3420 |    1.000 | 0.000/0.001
    40 |     11 |         9437 |        29640 |        29640 |    1.000 | 0.002/0.009
    80 |      9 |        58592 |       246480 |       246480 |    1.000 | 0.015/0.059
   160 |     17 |       451997 |      2009760 |      2009760 |    1.000 | 0.102/0.510
   320 |     13 |      3272936 |     16230720 |     16230720 |    1.000 | 0.794/4.164
  n  10 ->  20 : tests x6.37 (early exit)   x9.50 (full; theory x8)
  n  20 ->  40 : tests x7.09 (early exit)   x8.67 (full; theory x8)
  n  40 ->  80 : tests x6.21 (early exit)   x8.32 (full; theory x8)
  n  80 -> 160 : tests x7.71 (early exit)   x8.15 (full; theory x8)
  n 160 -> 320 : tests x7.24 (early exit)   x8.08 (full; theory x8)
  log-log slope of work: full = 3.08 ; early exit = 2.79
  PREDICTION CHECK: I predicted early exit would behave close to n^2 for random points.
  Observed slope 2.79 => prediction NOT supported; early exit lowers the constant (about 5x fewer
  tests at n=320) but the growth stays close to cubic.  Hypothesis: points are scanned in x-sorted
  order, so neighbouring points tend to lie on the same side of a line and a 'second side' is found late.
  Test of hypothesis: same random sets, but 'other points' scanned in RANDOM order:
    n=  10  tests sorted-scan=      208   random-scan=      154   (1.4x fewer)
    n=  20  tests sorted-scan=     1356   random-scan=      925   (1.5x fewer)
    n=  40  tests sorted-scan=     8648   random-scan=     3002   (2.9x fewer)
    n=  80  tests sorted-scan=    60080   random-scan=    11874   (5.1x fewer)
    n= 160  tests sorted-scan=   451213   random-scan=    95117   (4.7x fewer)
    n= 320  tests sorted-scan=  3377844   random-scan=   261732   (12.9x fewer)
    log-log slope with random scan order = 2.15

WORST-CASE FAMILY for the early-exit version: n points on a circle (every point is a hull vertex)
  n=  10 hull=  10 tests(early)=      227  n(n-1)(n-2)/2=      360  ratio 0.631
  n=  20 hull=  20 tests(early)=     1652  n(n-1)(n-2)/2=     3420  ratio 0.483
  n=  40 hull=  40 tests(early)=    12102  n(n-1)(n-2)/2=    29640  ratio 0.408
  n=  80 hull=  80 tests(early)=    91402  n(n-1)(n-2)/2=   246480  ratio 0.371
  n= 160 hull= 160 tests(early)=   707602  n(n-1)(n-2)/2=  2009760  ratio 0.352

CORRECTNESS ARGUMENT (geometric reasoning)
 Let H be the half-plane bounded by the line L through Pi and Pj that contains
 every other point (accepted edge => all s have the same sign or are 0).
 (a) A half-plane is convex, so it contains the convex hull of the whole set.
     Hence the hull lies entirely on one side of L.
 (b) Pi and Pj are points of the set, so they lie on L and belong to the hull;
     the hull is convex so the segment PiPj also lies in the hull.
 (c) A point of the hull that lies on a line that supports the hull (touches it
     with the whole hull on one side) cannot be an interior point: any
     neighbourhood of it would contain points on the empty side of L.  So
     PiPj lies on the boundary.  Conversely every hull edge lies on a
     supporting line, so such a pair passes the test: no edge is missed.
 If some Pk were on the other side, a segment from Pk to a point beyond L would
 cross L inside the hull, so PiPj would pass through the interior and could not
 be boundary - the test rejects exactly those pairs.
```

Execution output – Q6



```
Terminal  -  page 1/2

$ python3 q6_exhaustive_search.py

================================================================================
Q6  EXHAUSTIVE SEARCH - HUMANITARIAN AIRLIFT PLANNING
================================================================================

CORRECTNESS INSTANCE: 5 packages, payload limit 10 kg
  ID  Package      Weight  Value
  1   Medical kit       4     10
  2   Water             6      7
  3   Food              5      8
  4   Blankets          3      4
  5   Radio             2      6

  All 2^5 = 32 subsets (listed independently of the search code):
  mask   packages       weight  value  status
  00000  []                 0      0  feasible
  00001  [1]                4     10  feasible
  00010  [2]                6      7  feasible
  00011  [1, 2]            10     17  feasible
  00100  [3]                5      8  feasible
  00101  [1, 3]             9     18  feasible
  00110  [2, 3]            11     15  over limit
  00111  [1, 2, 3]         15     25  over limit
  01000  [4]                3      4  feasible
  01001  [1, 4]             7     14  feasible
  01010  [2, 4]             9     11  feasible
  01011  [1, 2, 4]          13     21  over limit
  01100  [3, 4]             8     12  feasible
  01101  [1, 3, 4]          12     22  over limit
  01110  [2, 3, 4]          14     19  over limit
  01111  [1, 2, 3, 4]       18     29  over limit
  10000  [5]                2      6  feasible
  10001  [1, 5]             6     16  feasible
  10010  [2, 5]             8     13  feasible
  10011  [1, 2, 5]          12     23  over limit
  10100  [3, 5]             7     14  feasible
  10101  [1, 3, 5]          11     24  over limit
  10110  [2, 3, 5]          13     21  over limit
  10111  [1, 2, 3, 5]       17     31  over limit
  11000  [4, 5]             5     10  feasible
  11001  [1, 4, 5]          9     20  feasible
  11010  [2, 4, 5]          11     17  over limit
  11011  [1, 2, 4, 5]       15     27  over limit
  11100  [3, 4, 5]          10     18  feasible
  11101  [1, 3, 4, 5]       14     28  over limit
  11110  [2, 3, 4, 5]       16     25  over limit
  11111  [1, 2, 3, 4, 5]     20     35  over limit

  Hand-checkable optimum from the table above = 20
```

Terminal - page 2/2

```
$ python3 q6_exhaustive_search.py

  Algorithm result: value=20, packages=[1, 4, 5] (Medical kit, Blankets, Radio), weight=9
  candidates examined=32 (=2^5)   feasible=17   -> correct: True
  Randomised cross-check on 200 instances (n<=12) against dynamic programming: mismatches = 0

PREDICTED GROWTH: candidates = 2^n ; +1 package doubles the search space

EXPERIMENT (random instances, payload = half of total weight)
   n |   examined        2^n |   feasible  item checks |  time (s) | t/t_prev | best value
   8 |        256        256 |        128         2048 |     0.000 |          | 290
  10 |       1024       1024 |        512        10240 |     0.001 |     4.55 | 270
  12 |       4096       4096 |       2067        49152 |     0.004 |     4.83 | 424
  14 |      16384      16384 |       8286       229376 |     0.017 |     4.81 | 558
  16 |      65536      65536 |      33006      1048576 |     0.082 |     4.72 | 802
  18 |     262144     262144 |     131072      4718592 |     0.356 |     4.35 | 765
  20 |    1048576    1048576 |     524288     20971520 |     1.655 |     4.65 | 762
  22 |    4194304    4194304 |    2112396     92274688 |     7.105 |     4.29 | 940

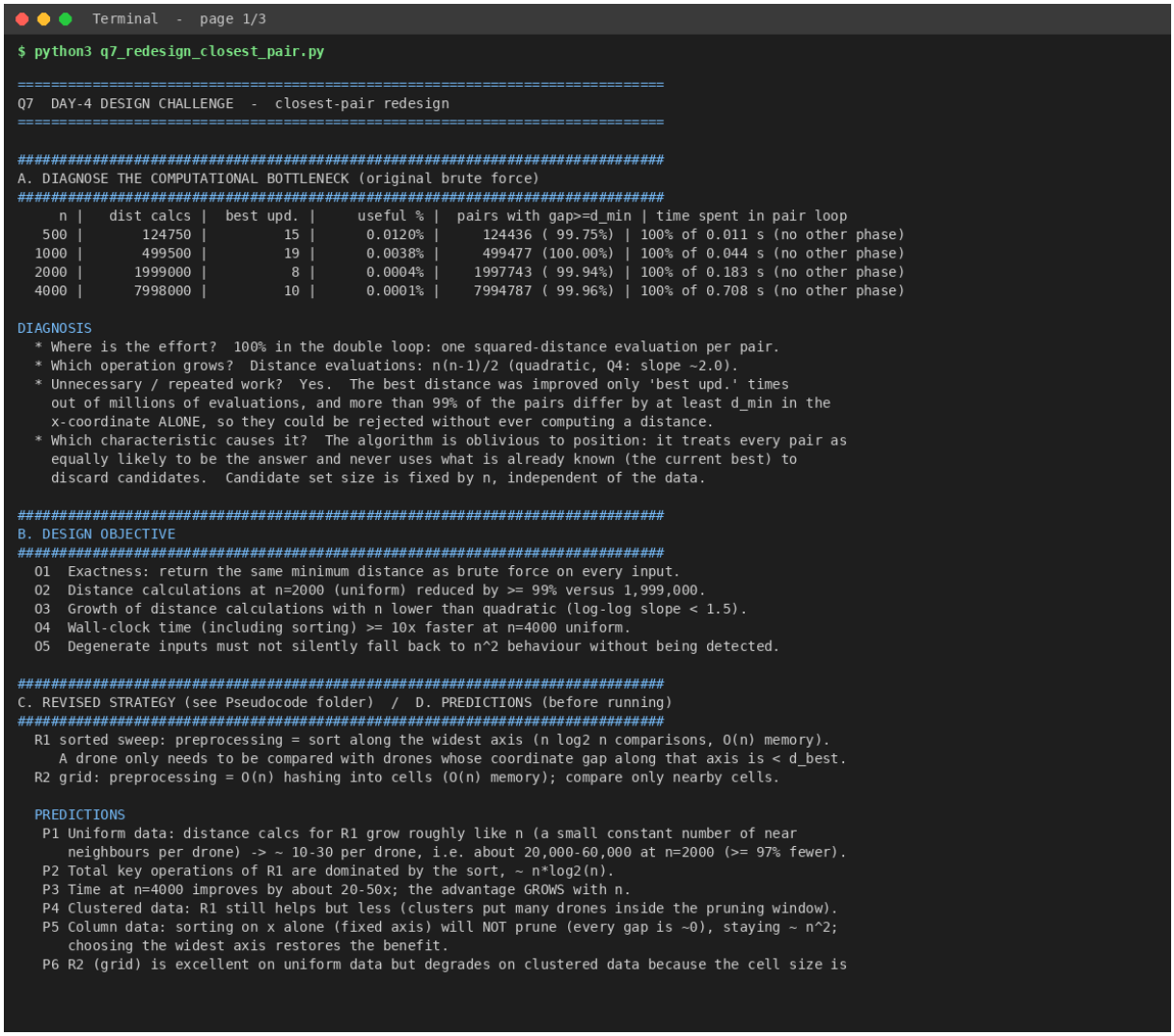
THEORY vs EXPERIMENT for +2 packages (candidates x4 predicted):
  n  8 -> 10 : candidates x4.0   item checks x5.00   time x3.50
  n 10 -> 12 : candidates x4.0   item checks x4.80   time x5.14
  n 12 -> 14 : candidates x4.0   item checks x4.67   time x4.81
  n 14 -> 16 : candidates x4.0   item checks x4.57   time x4.73
  n 16 -> 18 : candidates x4.0   item checks x4.50   time x4.35
  n 18 -> 20 : candidates x4.0   item checks x4.44   time x4.65
  n 20 -> 22 : candidates x4.0   item checks x4.40   time x4.29

  fitted model t(n) = c * n * 2^n with c = 7.708e-08 s ;   at n=22 cost per subset = 1.69 microseconds

DEPLOYMENT DECISION
  Fixed response time chosen: 10 s.  Reason: a loading plan is made before departure, the planner
  can wait a few seconds, but must be able to re-run it several times as packages / limit change;
  >10 s breaks that interactive 'what-if' loop during an emergency.
  Largest n with predicted t(n) <= 10 s  ->  n = 22 packages (this machine, interpreted Python)
  Safety margin (inputs vary, machine may be slower; pick 1 step lower): deploy for n <= 21

     n |     candidates 2^n |   predicted time | verdict
    20 |            1048576 |            1.6 s | OK
    22 |            4194304 |            7.1 s | OK
    24 |           16777216 |           31.0 s | too slow
    25 |           33554432 |           64.7 s | too slow
    26 |           67108864 |          2.2 min | too slow
    30 |         1073741824 |         41.4 min | too slow
    40 |      1099511627776 |        39.2 days | too slow
    50 |   1125899906842624 |   1.38e+02 years | too slow
    60 | 1152921504606846976 |   1.69e+05 years | too slow
  A 100x faster compiled implementation adds only log2(100) ~ 6-7 packages to the limit.
```

Execution output – Q7



Terminal - page 2/3

```
$ python3 q7_redesign_closest_pair.py

      tuned to average density (many drones per cell).
   P7 For very small n (< ~30) the original may be as fast or faster (sort overhead).

########################################################################
E. IMPLEMENT AND VALIDATE  (same test conditions: Q4 generator, same seeds)
########################################################################

DATASET: U  uniform random in 10 km x 10 km (same data as Q4)
     n |  BRUTE calcs |    R1 sort    R1 gaps   R1 calcs  R1 reduc | R2 lookups   R2 calcs |   t brute      t R1      t R2 | same answer
   100 |         4950 |        927        297        199    95.98% |        900        478 |   0.0004s   0.0001s   0.0003s | True
   250 |        31125 |       2650        671        422    98.64% |       2250       1271 |   0.0024s   0.0001s   0.0008s | True
   500 |       124750 |       5827       1223        727    99.42% |       4500       2613 |   0.0103s   0.0003s   0.0017s | True
  1000 |       499500 |      12628       1374        375    99.92% |       9000       5336 |   0.0423s   0.0004s   0.0035s | True
  2000 |      1999000 |      27291       5212       3214    99.84% |      18000      10593 |   0.1701s   0.0013s   0.0069s | True
  4000 |      7998000 |      58512       8010       4013    99.95% |      36000      21657 |   0.7112s   0.0024s   0.0143s | True
  8000 |     31996000 |     125113      16673       8674    99.97% |      72000      43204 |   2.8754s   0.0081s   0.0320s | True

DATASET: C  clustered (5 Gaussian clusters, sigma=150 m)
     n |  BRUTE calcs |    R1 sort    R1 gaps   R1 calcs  R1 reduc | R2 lookups   R2 calcs |   t brute      t R1      t R2 | same answer
   500 |       124750 |       5819        735        236    99.81% |       5020      22874 |   0.0109s   0.0002s   0.0041s | True
  1000 |       499500 |      12637       1516        517    99.90% |       9112      68004 |   0.0440s   0.0005s   0.0105s | True
  2000 |      1999000 |      27269       3191       1192    99.94% |      19512     151697 |   0.1783s   0.0010s   0.0241s | True
  4000 |      7998000 |      58506       8073       4074    99.95% |      37512     525548 |   0.7129s   0.0024s   0.0930s | True

DATASET: V  vertical corridor (x-spread 1 m, y-spread 10 km) - tests axis choice
     n |  BRUTE calcs |  R1 fixed x-axis calcs |   R1 widest-axis calcs | same answer
   500 |       124750 |          18503 ( 14.8%) |             12 (  0.0%) | True
  1000 |       499500 |         176709 ( 35.4%) |             17 (  0.0%) | True
  2000 |      1999000 |         255873 ( 12.8%) |             24 (  0.0%) | True
  4000 |      7998000 |         593180 (  7.4%) |             55 (  0.0%) | True

DATASET: W  two parallel vertical lines x=0 and x=10 km (x-spread 10 km > y-spread 5 km) - ADVERSARIAL for R1
     n |  BRUTE calcs | R1 calcs (% of brute) |        R2 grid calcs | same answer
   500 |       124750 |        62250 ( 49.9%) |         12145 (  9.7%) | True
  1000 |       499500 |       249500 ( 49.9%) |         34615 (  6.9%) | True
  2000 |      1999000 |       999000 ( 50.0%) |         97190 (  4.9%) | True
  4000 |      7998000 |      3998000 ( 50.0%) |        272339 (  3.4%) | True

EDGE CASES (correctness): duplicates, two points, identical x
  two drones           brute=5.000  sweep=5.000  grid=5.000  agree=True
  duplicate position   brute=0.000  sweep=0.000  grid=0.000  agree=True
  all same x           brute=1.000  sweep=1.000  grid=1.000  agree=True

CROSSOVER FOR SMALL n (uniform data, time per call, 300 repetitions)
     n |   brute (us) |   sweep (us) | faster
     5 |          1.4 |          3.2 | brute
    10 |          4.7 |          5.0 | brute
    20 |         15.5 |         11.9 | sweep
```

Terminal - page 3/3

```
$ python3 q7_redesign_closest_pair.py

    30 |         37.8 |         18.0 | sweep
    50 |        102.5 |         27.5 | sweep
   100 |        428.3 |         57.8 | sweep

OBJECTIVE / PREDICTION CHECK
  O1 exactness            : all runs agree with brute force -> met
  O2 reduction at n=2000  : 99.839% fewer distance calcs (1999000 -> 3214)  target >= 99%
  O3 growth exponent      : distance calcs slope = 0.86 ; total key ops (sort+gaps+calcs) slope = 1.06 (brute 2.00)
  O4 speed-up at n=4000   : R1 298.8x faster ; R2 49.7x faster
  P2 sort dominates?      : at n=2000 sort comps=27291, gap checks=5212, distance calcs=3214
```

Appendix B – Repository contents

```
CO2_Assignment/
├── Q1_Sorting_Algorithms/        Pseudocode/  Implementation_Code/
Output_Screenshots/ (+ raw_data/)
├── Q2_Sequential_Search/         ...
├── Q3_Brute_Force_String_Matching/
├── Q4_Brute_Force_Closest_Pair/
├── Q5_Brute_Force_Convex_Hull/
├── Q6_Exhaustive_Search/
├── Q7_Day4_Design_Challenge/
├── Final_Report/CO2_Assignment_Report.docx
├── tools/render_screenshot.py    run_all.py    README.md
```